Быстрое ядро для $D_n(G) = G \cap (1 + \Delta^n)$: профиль, вариант A, вариант B

docs/analysis.ru.md, ispacex/cflint-fast-core@8e276a1

Содержание

0. Итог . . . 1
1. Профиль текущего ядра (метод A, numpy) . . . 2
2. Вариант A: перенос на C . . . 2
3. Вариант B: $\mathbb{Z}[G]/\Delta^n$ как фактор усечённой свободной алгебры . . . 2
   3.1. Утверждение . . . 2
   3.2. Доказательство . . . 3
   3.3. Откуда брать соотношения . . . 5
   3.4. Все уровни за один раз . . . 5
   3.5. Проверка подгруппы-кандидата K (сито) . . . 5
   3.6. Где B проигрывает . . . 5
4. Замеры . . . 6
5. Сверка . . . 6
6. Рекомендации . . . 8
   6.1. Что запускать . . . 8
   6.2. Что делать с ядром дальше . . . 8
   6.3. Оценка для вопроса о плато [IMP26, §3.3]: $w \in D_7$? . . . 8
   6.4. Замер: размер базиса из стандартных мономов для группы плато . . . 8
   6.5. GPU-ядро плотной формы Хауэлла по модулю 2^{32} (прототип) . . . 10
   6.6. D5-7: поиск метабелевой группы класса 4 с $D_7 \neq 1$ (первый проход) . . . 11
   6.7. Класс 5 (D5-7): сдвиг плато [IMP26, §3.3] . . . 12
   6.8. D5-7, класс 4: пробел $D_7 \not\subseteq \gamma_4$ и все $K \subseteq \gamma_4(M)$ сразу . . . 13
   6.9. D5-7: кольцо L'_2 на четырёх образующих, проверка на компьютере; его групповой аналог . . . 16
   6.10. D5-7, класс 5: все $K \subseteq \gamma_5(M)$ сразу для сдвинутого плато . . . 17
   6.11. D5-9: на чём держится (3.12) и что нужно копредставлению класса 5 . . . 19
   6.12. D5-10: тождество носителя, почему он даёт один уровень и что нужно конструкции класса 5 . . . 23
   6.13. D5-11: J_n без плотной формы — стандартные мономы, уровень за уровнем . . . 27
7. Литература и известен ли метод . . . 30

Исходное ядро — эталонная реализация `dimsub.py` (метод A на numpy; она в отдельном, пока закрытом репозитории). Здесь — его профиль, перенос на C (вариант A), другой способ считать (вариант B) с доказательством корректности, сверка, замеры и рекомендации.

Обозначения: G — конечная p-группа, $x_1, \dots, x_d$ — её порождающие, $\Delta \subset \mathbb{Z}[G]$ — аугментационный идеал, F — свободная группа на $x_1..x_d$, $\mathfrak{f} \subset \mathbb{Z}[F]$ — её аугментационный идеал, $A_n = \mathbb{Z}\langle a_1, \dots, a_d\rangle/(\text{степень} \geq n)$ — усечённая свободная ассоциативная алгебра, A_n^+ — её часть без константы, $M = \operatorname{rk} A_n^+ = d + d^2 + \cdots + d^{n-1}$.

0. Итог

- **Вариант B верен** (доказательство — §3) и **подтверждён сверкой**: 0 расхождений с `dimsub.py` на всех группах, где есть ответы (§5), и полное совпадение с независимой реализацией того же метода в обе стороны.

- **Скорость.** На группах порядка 3^7: numpy 80–330 с, A на C 7–22 с (×11–15), B 0.2–1.4 с (×230–380, вместе с запуском Python). Все 1202 кандидата порядка 3^7 вариант B прошёл за 157 с на трёх потоках (у numpy — около 100 CPU-часов).
- **Главное — B не зависит от $|G|$.** Группа Бартольди–Михайлова порядка 3^{494} ($d = 7$): $\mathbb{Z}[G]/\Delta^4$ — за 15 с по модулю 3^{19}, точный HNF над $\mathbb{Z}$ — 510 с, индексы совпадают. Рипсовские 2-группы порядков 2^{40}–2^{107} — секунды на группу.
- **Предел B** — $M = \sum_{k<n} d^k$: растёт как d^{n-1}. При $d = 6$, $n = 7$ (вопрос о плато из [IMP26, §3.3]) $M = 55\,986$, и плотная линейная алгебра там не годится (оценка — §6.3).
- **FLINT `nmod_mat_howell_form`** на этих матрицах медленнее даже numpy (§2); своя форма Хауэлла для локального кольца $\mathbb{Z}/p^E$ — в 10–15 раз быстрее numpy.

1. Профиль текущего ядра (метод A, numpy)

Метод A: $\Delta \cong \mathbb{Z}^{|G|-1}$, $\Delta^n = \mathrm{span}\{b(x_i - 1)\}$ по базису b решётки Δ^{n-1}, форма Хауэлла по модулю p^E ($E = 19$ для $p = 3$, 30 для $p = 2$), цикл по столбцам на Python, строки обрабатываются векторно.

Замер `scripts/profile_dimsub.py` на тестовой машине, один процесс под `nice -n 10`. Машина общая с CI, средняя загрузка во время замеров 15–29 при 16 потоках, поэтому цифры шумят на десятки процентов.

группа	d	матрица на уровень	howell по уровням $n = 2, 3, 4, 5$, с	проверка $g \in D_n$, с	всего, с
2187 #171	2	4372×2186	4.9 / 14.3 / 22.2 / 28.9	1.8	73.5
2187 #4721	3	6558×2186	62.3 / 37.4 / 67.7 / 76.2	8.0	253.6
2187 #9066	4	8744×2186	74.4 / 47.2 / 86.1 / 91.2	6.7	307.9
512 #864	2	1022×511	0.34 / 0.24 / 0.39	0.06	1.1
512 #405386	4	2044×511	0.94 / 0.81 / 1.31	0.08	3.3

- **Всё время — форма Хауэлла**, 95–98 %. Построение блоков $b \cdot (x_i - 1)$ занимает доли секунды, проверка принадлежности — единицы процентов.
- Матрица на каждом уровне одного размера, $d(|G| - 1) \times (|G| - 1)$, ранг всегда $|G| - 1$. Стоимость $\sim d|G|^3/2$ операций на уровень. Рост времени с n даёт p-кручение Δ/Δ^n: строки $p^{E-v}\cdot$(опорная) возвращаются в пул.
- В исходном прогоне выборки 2187 (14 процессов параллельно) медиана была 278 с, максимум 659 с. Один процесс быстрее в 2–4 раза: при полной загрузке numpy упирается в память.
- Порядок $|G|^3$ — главный предел: 2^{10} на уровень в 8 раз дороже 512, а группу порядка 2^{38} перечислить нельзя вовсе.

2. Вариант A: перенос на C

`c/dimsub_a.c` и `c/hw.h`: тот же алгоритм, те же E, тот же вывод, что у `dimsub.analyse`, в быстром режиме и с `--full`. Проверка «Δ^n — правый идеал» из `--full` не перенесена.

- **Своя форма Хауэлла** (по умолчанию). Кольцо $\mathbb{Z}/p^E$ локальное, поэтому опорной берётся строка с минимальной p-адической оценкой в столбце, и она делит остальные: одно вычитание на элемент, без `xgcd` на парах строк. Строки хранятся как `uint32` ($p^E < 2^{31}$), `row -= f·piv` считается в `uint64` с редукцией через `double` ($q = \lfloor x/D \rfloor$ с ошибкой ≤ 1 для $x < 2^{63}$, одна поправка), без деления. Цикл векторизуется (`-O3 -march=native`, на тестовой машине есть AVX-512DQ).
- **FLINT** (`--flint`): `nmod_mat_howell_form` из FLINT 3.3 (conda-forge `libflint`). Она общая для $\mathbb{Z}/N$: на каждую пару строк — `xgcd` и унимодулярное преобразование 2×2; кроме того, требует строк не меньше, чем столбцов. На наших матрицах она **медленнее numpy** (таблица в §4).
- Отложенная редукция (копить несколько `row -= f·piv` без остатка) при $p^E \approx 2^{31}$ в 64 бита не помещается: каждое произведение уже $\approx 2^{62}$. Она имеет смысл только при меньшем E (§6).

3. Вариант B: $\mathbb{Z}[G]/\Delta^n$ как фактор усечённой свободной алгебры

3.1. Утверждение

Пусть $G = F/N$, $\pi : \mathbb{Z}[F] \to \mathbb{Z}[G]$, $\mathfrak{r} = \ker\pi$, и $\mu : \mathbb{Z}[F] \to A_n$ — кольцевой гомоморфизм Магнуса $x_i \mapsto 1 + a_i$ (тогда $x_i^{-1} \mapsto \sum_{k<n}(-a_i)^k$). Пусть $R_0 \subset F$ нормально порождает N.

Теорема. $\mathbb{Z}[G]/\Delta^n \cong A_n/J_n$, где $J_n = \operatorname{span}_{\mathbb{Z}}\{\, u\,(\mu(r)-1)\,v : r \in R_0;\ u, v$ — мономы$\}$ — двусторонний идеал, порождённый элементами $\mu(r)-1$. Изоморфизм переводит g в $\mu(w_g)$ для любого слова w_g, представляющего g. Поэтому

$$g \in D_n(G) \iff \mu(w_g) - 1 \in J_n.$$

Кроме того, $A_n^+/J_n \cong \Delta/\Delta^n$ — конечная p-группа, а $\operatorname{rk} A_n^+ = M$ не зависит от $|G|$.

3.2. Доказательство

(a) Усечённый изоморфизм Магнуса: $\mathbb{Z}[F]/\mathfrak{f}^n \cong A_n$**.** μ определён: элемент $1 + a_i$ обратим в A_n (обратный — $\sum_{k<n}(-a_i)^k$, так как $a_i^n = 0$), F свободна, поэтому $x_i \mapsto 1 + a_i$ продолжается до гомоморфизма $F \to A_n^\times$, а линейно — до кольцевого $\mathbb{Z}[F] \to A_n$. Константа $\mu(f)$ равна аугментации $\varepsilon(f)$: оба отображения — кольцевые гомоморфизмы в $\mathbb{Z}$, совпадающие на x_i. Поэтому $\mu(\mathfrak{f}) \subseteq A_n^+$ и $\mu(\mathfrak{f}^n) \subseteq (A_n^+)^n = 0$, и получается $\bar\mu : \mathbb{Z}[F]/\mathfrak{f}^n \to A_n$. Обратно, по универсальному свойству свободной ассоциативной алгебры есть кольцевой $\varphi : \mathbb{Z}\langle a\rangle \to \mathbb{Z}[F]/\mathfrak{f}^n$, $a_i \mapsto x_i - 1$. Моном степени $\geq n$ переходит в произведение n элементов $\mathfrak{f}$, то есть в 0, так что φ пропускается через A_n. Далее, $\bar\mu\varphi(a_i) = a_i$, значит $\bar\mu\varphi = \mathrm{id}$. И $\varphi\bar\mu(x_i) = x_i$, значит $\varphi\bar\mu$ тождественно на F (на x_i^{-1} — потому что кольцевой гомоморфизм сохраняет обратные), а F аддитивно порождает $\mathbb{Z}[F]$. □

Глубокая теорема Магнуса ($\mu^{-1}(\deg \geq n) = \mathfrak{f}^n$ в $\mathbb{Z}\langle\langle a\rangle\rangle$) здесь не нужна: для усечения это утверждение элементарно.

(b) $\ker\pi = \mathfrak{r}$ **— двусторонний идеал** I**, порождённый** $\{r - 1 : r \in R_0\}$**.** Включение $I \subseteq \ker\pi$ очевидно. Для $n \in N$ имеем $n - 1 \in I$, так как $wrw^{-1} - 1 = w(r-1)w^{-1}$, $r^{-1} - 1 = -r^{-1}(r-1)$ и $ab - 1 = a(b-1) + (a-1)$ — индукция по длине n как произведения сопряжённых $r^{\pm1}$. Наконец, $\ker\pi \subseteq \mathbb{Z}[F](N-1)$: если $\sum c_f f \in \ker\pi$, то в каждом смежном классе tN сумма коэффициентов нулевая, и $\sum_{f\in tN} c_f f = t\sum c_f(t^{-1}f - 1)$ с $t^{-1}f \in N$. □

(b′) Если S порождает N **как подгруппу** (например, S — порождающие Шрайера), то $\ker\pi = \sum_{s\in S}(s-1)\mathbb{Z}[F]$, и достаточно **правого** идеала: $ns - 1 = (n-1)s + (s-1)$, $s^{-1} - 1 = -(s-1)s^{-1}$, а $\ker\pi = (N-1)\mathbb{Z}[F]$ по той же выкладке с правыми смежными классами.

(c) Сборка. $\pi(\mathfrak{f}) = \Delta$ (образ аугментационного идеала при сюръекции групповых колец), значит $\pi(\mathfrak{f}^n) = \Delta^n$ и $\mathbb{Z}[G]/\Delta^n \cong \mathbb{Z}[F]/(\mathfrak{r} + \mathfrak{f}^n)$. Под $\bar\mu$ подмодуль $(\mathfrak{r} + \mathfrak{f}^n)/\mathfrak{f}^n$ переходит в $\mu(\mathfrak{r})$. Образ идеала при сюръективном кольцевом гомоморфизме — идеал, порождённый образами порождающих, то есть $\mu(\mathfrak{r}) = J_n$. Как абелева группа он натянут на $u\,\mu(r-1)\,v$ с мономами u, v, потому что A_n аддитивно порождается мономами. Наконец, $g - 1 \in \Delta^n \iff w_g - 1 \in \mathfrak{r} + \mathfrak{f}^n \iff \mu(w_g) - 1 \in J_n$. □

(d) Конечность и точность по модулю p^E**.** $\varepsilon(J_n) = 0$, так что $A_n = \mathbb{Z} \oplus A_n^+$ и $A_n^+/J_n \cong \Delta/\Delta^n$. Умножение даёт эпиморфизм $(\Delta/\Delta^2)^{\otimes k} \to \Delta^k/\Delta^{k+1}$, а $\Delta/\Delta^2 \cong G_{ab}$ — конечная p-группа. Значит, все Δ^k/Δ^{k+1} и сама Δ/Δ^n — конечные p-группы. Поэтому **сертификат точности из dimsub.py переносится дословно**. Непринадлежность $\mu(w_g) - 1 \notin J_n + p^E A_n^+$ точна всегда. Принадлежность точна, если $p^{E-1}A_n^+ \subseteq J_n + p^E A_n^+$: тогда показатель A_n^+/J_n делит p^{E-1} и $p^E A_n^+ \subseteq J_n$. Кроме того, $\log_p |A_n^+/(J_n + p^E A_n^+)| = \log_p |\Delta/(\Delta^n + p^E\Delta)|$, так что поле `index_logp` обязано совпасть с dimsub.py. В сверке оно совпадает.

(e) Сертификат ранга для бесконечной G **(D5-21, пункт 6).** Для бесконечной нильпотентной G $A_n^+/J_n \cong \Delta/\Delta^n \cong \mathbb{Z}^R \oplus T$ с конечной T. Проверке точности в `cert/check_in.c` нужно $R \geq |F|$, где F — столбцы без ведущего элемента по модулю p^E ($R \leq |F|$ даёт вторая половина (X)). Раньше эта оценка бралась из теоремы Квиллена ($R = r_n(h)$) с рангами h факторов γ_k/γ_{k+1} из nq; и то и другое было на доверии. Теперь она сертифицирована. *Полиномиальное отображение степени* $\leq w$ на G — это $\mathbb{Z}$-линейное $\varphi : \mathbb{Z}[G] \to \mathbb{Z}$ с $\varphi(\Delta^{w+1}) = 0$ (Пасси).

1. Функционал λ на A_{w+1}^+ задаёт $\varphi(g) = \lambda(\mu(w_g) - 1)$. Это полиномиальное отображение степени $\leq w$ тогда и только тогда, когда $\lambda(J_{w+1}) = 0$, т. е. $\lambda(u(\mu(r)-1)v) = 0$ для соотношений r и мономов u, v с $\deg u + \deg v \leq w - 1$ (по (c)). Это конечный набор целочисленных тождеств.
2. Если степени φ, ψ не больше a, b, то поточечное произведение $\varphi\psi$ имеет степень $\leq a + b$: $\varphi\psi = (\varphi \otimes \psi) \circ \delta$, где $\delta(g) = g \otimes g$. Из $\delta(g-1) = (g-1)\otimes 1 + 1 \otimes (g-1) + (g-1)\otimes(g-1)$ следует $\delta(\Delta^N) \subseteq \sum_{i+j\geq N} \Delta^i \otimes \Delta^j$.
3. Полиномиальное отображение степени $< n$, равное нулю в 1, — функционал на Δ/Δ^n. Если k таких линейно независимы, то $R \geq k$. Независимость — это ранг матрицы их значений на положительных словах s длины $1, \ldots, n-1$: элементы $\mu(s) - 1$ образуют базис A_n^+, унитреугольный относительно мономов. Ранг по модулю простого — нижняя оценка ранга над $\mathbb{Q}$.

Итак, сертификат — список функционалов λ_i на $A_{w_i+1}^+$. `cert/polymaps.h` проверяет для каждого пункт 1 в целых числах: по модулю стольких 31-битных простых, сколько требует оценка $|\lambda_i(u(\mu(r)-1)v)|$ в плавающей точке. Затем он строит все произведения суммарного веса $< n$ по пункту 2, точно вычисляет их значения на словах и находит их

ранг k по модулю простого по пункту 3. Точность после этого требует $k = |F|$. Проверяльщик трасс `cert/trace`, не делящий кода с `cert/polymaps.h`, повторяет проверки 1–3, причём оценку для проверки 1 считает в точных целых. Функционалы пишет GAP (`gap/polymaps.g`, через `cert/make_polymaps.py`): это показатели pc-порождающих бесконечного порядка группы G/γ_{W+1} из nq, вес — глубина в нижнем центральном ряду (координаты Мальцева вдоль изоляторов — опция `--mode iso`). Произведений веса $< n$ ровно $r_n(h)$ (Дженнингс, Квиллен), но ничего из этого не принимается на веру. Отображение неверной степени отвергается, а при недостатке независимых произведений не проходит точность. `cert/test_polymaps.py` показывает, что несло h. В $C_{2^{30}}$ с $h = 1$ (истинное h равно 0) путь через Квиллена принимает ложное $x^{2^{24}} \in D_n$ по модулю 2^{20}: там кручение порядка 2^{30} выглядит свободным. Полиномиальное отображение отвергается, потому что $\lambda(\mu(x^{2^{30}}) - 1) = 2^{30} \neq 0$.

(f) Трассы принадлежности и Lean (D5-21, пункты 1 и 7). Каждое утверждение о принадлежности записано ещё и как *трасса* (`cert/make_trace.c`, без доверия): программа без ветвлений, каждый вектор которой равен $u \cdot$ (порождающий $+ \sum_i c_i V_i$), где порождающие — $\mu(r) - 1$, усечённые до A_k, и сдвиги $X_j V$, $V X_j$ вектора уровня $k - 1$; затем по свидетелю на каждый член ($\mu(w) - 1 = -\sum_i c_i V_i$) и на каждый столбец вне свободных столбцов F для проверки точности (X) из (d), причём оценку ранга $R \geq |F|$ дают полиномиальные отображения из (e). Проверяльщик `cert/trace` (Rust) не делит кода ни с `cert/check_in.c`, ни с производителем, ничего не выбирает (пересчитывает каждую строку и сравнивает) и сам проверяет отображения. Все 63 утверждения с принадлежностями получают VERIFIED (`cert/trace_all.sh`, 4 мин на одном ядре): §5 (группы типа Рипса, $\alpha \in D_4$ для 2^{84}, 2^{90}), §6.4 (4), §6.6 (25), §6.8 (7; у L1, L1b128, L1b256, L1free, L1x2free члены — строки коммутаторов `test_coeffs`, которые проверяльщик строит сам, предварительно точно проверив, что коммутаторы лежат в $\gamma_c(F)$; Лемма 3 и то, что строки — заявленные GAP элементы, принимаются), §6.10 (2), §6.12 (17) и §6.13 (6). Самая большая трасса, `c5i5` при $n = 8$, — 155 187 векторов, около минуты. Lean (group-dimension, `GroupDimension/TraceReplay/`; без `sorry`, аксиомы только `propext`, `Classical.choice`, `Quot.sound`) доказывает смысл проверки — для собственной её записи (`Check.lean`): если трасса проходит, а $g_1, \ldots, g_d$ порождают G и удовлетворяют использованным в трассе соотношениям, то каждый тест лежит в $D_n(G)$ (каждый lintest $\sum_k c_k(w_k - 1)$ — в Δ^n), когда свободных столбцов нет и экспонента G равна p^N, или, при свободных столбцах, когда отображения — гомоморфизмы $G \to \mathbb{Z}$ (вес 1), а тест имеет p-примарный порядок в Δ/Δ^n; эти гипотезы проверяет читатель. Две трассы проигрывает само ядро Lean (`decide +kernel`, без `native_decide`, после переноса `trace_to_lean.py`): $\alpha \in D_4$ для группы типа Рипса 2^{90} (§5) и `carrier_c4aT4` при $n = 4$ (§6.12; все 26 членов, из них 21 lintest; для трёх lintests гипотеза о порядке — на сам элемент). Ещё 28 утверждений подпадают под те же теоремы, но проверены только Rust-проверяльщиком: остальные 4 без отображений (группа типа Рипса 2^{84}, а также Rd46f15, Rf15, Rf16 из §6.6) и остальные 24 с отображениями веса 1 (§6.10, §6.12, §6.13); в Lean они не перенесены, и большинство их трасс слишком велики для ядра (при $n = 5$ `c4m4_L1free` с 1 130 векторами требует больше 17 ГБ; tm `gd-scale-m4`, 03.11.2026). Остальные 33 (§6.4: 4, §6.6: 22, §6.8: 7) используют отображения веса ≥ 2. Для них Lean (#25) доказывает правило произведения ((e), пункт 2, `lin_mul_eq_zero`) и теорему о принадлежности в заданной группе $G = \langle x \mid R \rangle$ (`check_mem_D_poly_pres`; линтесты через `check_mem_D_poly_lin` с `isBase_of_vanish`): если трасса проходит, соотношения трассы входят в R, сертификат (отображения, все веса $< n$, их произведения и точный целочисленный левый обратный на свободных столбцах) проходит, каждое отображение проходит проверку 1 из (e) на всех соотношениях R, а тест имеет порядок степени p в Δ/Δ^n, то тест лежит в $D_n(G)$ (линтест — в Δ^n). Для 31 из 33 (все, кроме двух утверждений `c4m4_P6`) ядро проверяет и саму часть с полиномиальными отображениями (group-dimension #26, `PolyData`, сборка `lake build PolyData`):

- проверку 1 из (e) для каждого отображения на всех соотношениях копредставления (`checkVanish`, или `checkVanishB` с биномиальными степенями для Rd4, Rd6, Rd46; корректность — `vanish_of_check`, `vanish_of_checkB`; $\mu(r)$ точно над $\mathbb{Z}$ ниже степени $w + 1$, длинные соотношения как степени u^k; всего 22 491 соотношение, различных в пределах каждого из пяти наборов отображений);
- точный целочисленный левый обратный на свободных столбцах трассы (`checkPolyF`). Проверка 3 Rust-чекера даёт лишь ранг по модулю $2^{31} - 1$.

Для этих 31 утверждения вне ядра остаются:

- сама трасса (331–13 858 векторов, только `cert/trace`);
- перенос F и соотношений (`poly_to_lean.py`);
- условие на порядок.

У двух утверждений `c4m4_P6` отображения веса 4, и одна ячейка их проверки 1 не помещается в 17 ГБ задачи.

3.3. Откуда брать соотношения

Из таблицы умножения (`py/magnus.py`; вход — та же JSON-строка, что у dimsub.py). Поиск в ширину по графу Кэли даёт остовное дерево и слова w_g; каждому ребру (g, x_i) сопоставляется

$$\rho_{g,i} = \mu(w_g)(1 + a_i) - \mu(w_{gx_i}) = \big(\mu(s_{g,i}) - 1\big)\,\mu(w_{gx_i}), \qquad s_{g,i} = w_g x_i w_{gx_i}^{-1}.$$

Элементы $s_{g,i}$ — порождающие Шрайера ядра N (порождают его как подгруппу), а $\mu(w_{gx_i})$ обратим. По (b′) $J_n = \mathrm{span}\{\rho_{g,i}\,v\}$: достаточно замыкания относительно умножения **справа** на a_j, а это сдвиг координат. Обратные элементы не нужны, все $\mu(w_g)$ считаются по дереву за $O(|G| \cdot M)$. Соотношений $(d-1)|G| + 1$, но столбцов всего M, и форма Хауэлла набирается порциями.

Из pc-копредставления, без перечисления группы (`gap/export_pres.g` → `py/magnus_pres.py`). GAP строит pc-последовательность, уточняющую нижний центральный ряд. Слой k — модулярный pcgs γ_k/γ_{k+1}, набранный просеиванием из коммутаторов $[b, x_j]$ (b — элемент слоя $k-1$) и их p-х степеней, так что у элемента слоя k слово $W \in \gamma_k(F)$. Соотношения такие: степенные $W_i^p = \prod W_j^{e_j}$, коммутаторные $[W_j, W_i] = \prod W_k^{e_k}$ и $x_j = \prod W_k^{e_k}$. Это копредставление G на $x_1..x_d$ по Тице: отображения $G = \langle s \mid \mathrm{pc}\rangle \to \Gamma$, $s_i \mapsto W_i$, и $\Gamma \to G$ взаимно обратны, что проверяется прямо на s_i и на x_j.

Отсечение по весу. $\mu(\gamma_k(F)) \subseteq 1 + A^{\geq k}$ — индукция по k из тождества $[x,y] - 1 = x^{-1}y^{-1}\big((x-1)(y-1) - (y-1)(x-1)\big)$. Поэтому соотношение из $\gamma_n(F)$ даёт $\mu(r) - 1 = 0$ в A_n и его можно не выписывать. Отбрасываются степенные соотношения слоёв $\geq n$ и коммутаторные с $k_i + k_j \geq n$: их правые части лежат в $\gamma_{k_i+k_j}(G)$, то есть в слоях $\geq k_i + k_j$. По той же причине слова элементов слоёв $\geq n$ заменяются единицей. Без этого у группы 3^{494} слова глубоких слоёв не помещаются в 8 ГБ.

3.4. Все уровни за один раз

J_n — образ J_N ($n < N$) при отбрасывании степеней $\geq n$. Если столбцы упорядочены по возрастанию степени, то строки формы Хауэлла J_N с опорным столбцом степени $< n$, усечённые до степеней $< n$, образуют форму Хауэлла J_n. Действительно, элемент проекции, равный нулю до столбца c, поднимается в J_N с теми же первыми координатами и раскладывается по строкам с опорой $\geq c$. То же верно для HNF над $\mathbb{Z}$. `magnus_pres.py` считает $J_{n_{\max}}$ один раз.

3.5. Проверка подгруппы-кандидата K (сито)

Если $K \subseteq D_{n-1}$ — элементарная абелева p-подгруппа с независимыми порождающими $g_1..g_r$, то $g \mapsto (g-1) \bmod \Delta^n$ на K — гомоморфизм: $gh - 1 = (g-1) + (h-1) + (g-1)(h-1)$ и $(g-1)(h-1) \in \Delta^{2n-2} \subseteq \Delta^n$. Значит, $D_n \cap K$ — ядро $c \mapsto \sum c_i v_i$ по модулю J_n, $v_i = \mu(w_{g_i}) - 1$. Это одна форма Хауэлла матрицы $[J_n \mid 0;\ V \mid I;\ 0 \mid pI]$: строки с опорой в последних r столбцах порождают ядро (`magnus_pres.py --kernel`). Нужны только слова для $g_1..g_r$, а не для всех элементов группы.

3.6. Где B проигрывает

Столбцов у B $M = \sum_{k=1}^{n-1} d^k$, у A — $|G| - 1$. В выборке порядка 3^7 ($d \leq 4$, $n \leq 5$) $M \leq 340$ против 2186; у выживших 512 ($d \leq 5$, $n \leq 4$) $M \leq 155$ против 511. Но при большом d и большом n ранг свободной алгебры обгоняет $|G|$: у C_2^6 при $n = 6$ $M = 9330$, у групп порядка 2^{10} с $d = 6$ при $n = 5$ $M = 1554 > 1023$. Отсюда два правила.

- В полном режиме сверки (все уровни до $n = 6$) B на группах с $M > 500$ останавливался и сверялся по общему префиксу уровней (поле `truncated`; число таких групп — в §5).
- На каждую группу выбирать B, если $M < |G|$, и A иначе.

Настоящее лекарство — хранить не плотную A_n, а только «стандартные» мономы. При упорядочении, согласованном с умножением (deg-lex), моном, ведущий в строке J с обратимым коэффициентом, остаётся ведущим с обратимым коэффициентом и во всех кратных $u\,w\,v$, то есть исключается вместе с ними. Остаётся порядок-идеал мономов, по размеру не меньше $\dim_{\mathbb{F}_p} \Delta_{\mathbb{F}_p}/\Delta^n_{\mathbb{F}_p}$ (§6.4). Это алгоритм нильпотентного фактора ассоциативной алгебры, но над кольцом $\mathbb{Z}/p^E$, а не над полем, как в ModIsom. В прототип он не вошёл (§6).

4. Замеры

Одна группа, один процесс, `nice -n 10`, тестовая машина под нагрузкой CI (`scripts/bench.sh`), время в секундах. У «B (Python)» около 0.15 с — запуск интерпретатора.

группа	d	numpy A (dimsub.py)	A на C	A на C, FLINT	B (Python)	A C / numpy	B / numpy
2187 #171	2	79.4	7.0	111.2	0.21	×11	×380
2187 #4721	3	263.1	14.8	248.5	0.75	×18	×350
2187 #9066	4	332.4	22.4	272.6	1.42	×15	×230
512 #864	2	1.21	0.11	1.01	0.21	×11	×6
512 #405386	4	2.51	0.21	2.71	0.31	×12	×8

Целые наборы (по три потока, стена):

набор	групп	B (Python)	A на C	numpy в исходном прогоне, CPU
порядки 8–625, полный режим (8 наборов)	2 762	509 с	9.1 с	—
729, все	504	9.2 с	25.7 с	13 780 с
256, класс ≥ 3	24 328	1 196 с	131 с	44 370 с
512, выжившие после сита	2 792	88 с	65 с	23 732 с
2187, выборка	49	7.4 с	248 с	17 198 с
2187, все кандидаты	1 202	157 с	4 905 с	$\approx$ 420 000 с (оценка, $\approx$ 117 ч)

На малых порядках (256, 512) A на C быстрее Python-варианта B: там M сравним с $|G|$ (у групп порядка 256 с $d = 6$ при $n = 5$ $M = 1554 > 255$), а у Python есть постоянные издержки на группу. На порядке 2187 B быстрее A на C в 30 раз, а numpy — в сотни раз.

Большие группы, только B по копредставлению:

группа	$\lvert G\rvert$	d	n	соотношений после отсечения	M	время
Бартольди–Михайлов [BM23]	3^{494}	7	4	6045	399	15 с mod 3^{19}; 510 с точн
рипсовское семейство (`gap/xcheck_fam.g`)	2^{40}–2^{107}	4	5	380–2330	340	1.7–9 с на группу

У группы 3^{494} индексы $\log_3 |\Delta/\Delta^n| = 59, 321, 1430$ ($n = 2, 3, 4$) совпали по модулю и над $\mathbb{Z}$. На 394 проверенных элементах (pcgs слоёв 2–4) $D_n = \gamma_n$ при $n \leq 4$, что согласуется с теоремой Пасси ($D_4 = \gamma_4$ для нечётных p).

5. Сверка

Эталон — ответы `dimsub.py` (хранятся вместе с ним, в отдельном репозитории). Сравниваются по уровням $|D_n|$, $|\gamma_n|$, `equal`, множество лишних элементов, `index_logp` и `certified`, где он посчитан в обоих.

Вариант B по перестановкам (`py/magnus.py`) и A на C (`c/dimsub_a`) — все эталонные наборы:

набор	групп	уровней B	уровней A	расхождений B / A
8, 16, 32, 64, 81, 128, 243, 625 — полный режим	2 762	14 961	16 575	0 / 0
729 — все	504	1 946	1 946	0 / 0
256 — класс ≥ 3	24 328	100 691	100 720	0 / 0
512 — выжившие после сита	2 792	11 168	11 168	0 / 0
2187 — выборка	49	245	245	0 / 0
всего	**30 435**	**129 011**	**130 654**	**0 / 0**

Совпадают $|D_n|$, $|\gamma_n|$, лишние элементы, сертификаты и индексы $\log_p[\Delta : \Delta^n + p^E\Delta]$. B в полном режиме на 1 425 группах (почти все элементарно-абелевы или близкие, $M > 500$) остановлен потолком ранга и сверен по префиксу уровней. В быстром режиме такое случилось у 29 групп порядка 256 с $d = 6$. A сверен целиком.

Вариант B по копредставлению (`gap/export_pres.g` → `py/magnus_pres.py`), проверены все элементы каждой группы: порядки 64, 128, 243, 729, 512 (выжившие), 2187 (выборка) — 6 007 групп, 17 140 уровней, 0 расхождений, включая индексы. Отдельно: 20 групп порядка 64 точным HNF над $\mathbb{Z}$ (`--exact`) и 5 групп порядка 2187 с $n_{\max} = 4 < \mathrm{cls} + 1$ (проверка отсечения по весу) — 0 расхождений.

Перекрёстная сверка с независимой реализацией того же метода (не опубликована; словари мономов, явный перебор пар u, v, соотношения G плюс все коммутаторы веса 4).

- Их код на наших группах (`scripts/xcheck_independent.py`; соотношения и слова из нашего экспорта) — те же 6 007 групп и 17 140 уровней, 0 расхождений, включая индексы.
- Наш код на их 14 рипсовских группах $Q = G/\gamma_4(G)$ порядков 2^{40}–2^{107} (`gap/xcheck_fam.g`). Копредставление строится заново из pc-группы Q, а не из копредставления G, так что общих у реализаций только метод и алгоритм Хауэлла.
  - $D_5 \cap \Omega_1(\gamma_3 \cap M_5) = 1$ во всех 14 группах — как у них.
  - $D_4 \cap \Omega_1(\gamma_3 \cap M_4)$ имеет ранг 1 ровно у 2^{84}, 2^{90}, 2^{107} и тривиально у остальных — как у них.
  - $\alpha = [e_0, e_1]^{32}[e_0, e_2]^{64}[e_1, e_2]^{128} \in D_4$ у 2^{84} и 2^{90}; у 2^{107} и двух других групп с $\alpha \neq 1$ — $\alpha \notin D_4$; у прочих $\alpha = 1$. Всё — как у них.
  - **Сертификат точности** ($p^{E-1}A_n^+ \subseteq J_n + p^E A_n^+$) проходит во всех 42 случаях. Поэтому принадлежности по модулю 2^{30} здесь точные, и $D_4(Q) \neq 1$ **для** 2^{84}**,** 2^{90} **и** 2^{107} **доказано вычислением**, а не только как «кандидат по модулю».

[компьютер, сертифицировано] (D5-21, 02.11.2026). Две части этого раздела теперь проверяет `cert/check_in.c`, не делящий кода ни с одним из вариантов:

- **SmallGroups против dimsub.py.** Для каждой группы эталонных наборов порядков 8–128, 81, 243, 625, 729, 2 792 «выживших» порядка 512 и 49 групп порядка 2187 (6 107 групп) GAP выгружает группу с **каждым элементом** как тестом (`gap/export_pres.g`), и проверяльщик решает принадлежность каждого при всех $n \leq \min(\mathrm{cls} + 1, 6)$; для этих конечных p-групп проверка точности (X) проходит на каждом уровне, так что $|D_n(G)|$ считается точно. Оно совпадает с $|D_n|$ из dimsub.py (значит, $D_n = \gamma_n$, как в эталоне), индекс тоже, на всех **17 344 уровнях**: 0 расхождений (`scripts/mass_smallgroups.sh`, `cert/mass_smallgroups.py`; таблицы по уровням — в `results/mass/`; CPU-контейнер Modal с `smallgrp-extra` для GAP; эталонные CSV опознаются по sha256 в `results/mass/*.out`). Где эталонный индекс помечен «?» (так dimsub.py отмечает несертифицированный индекс, 10 014 уровней), совпадение — лишь проверка согласованности. На доверии: SmallGroups в GAP и выгрузка (`export_pres.g`), а для «$D_n = \gamma_n$» — эталонные $|\gamma_n|$ (GAP). Порядок 256 (класс ≥ 3, 24 328 групп) не запускался; группы класса ≥ 6 проверены до $n = 6$.
- **Рипсовские группы** (`gap/xcheck_fam_run.g`, перестроены; записи `results/rips/`, утверждения `results/cert/rips/`, 33 штуки): $D_5 \cap \Omega_1(\gamma_3 \cap M_5) = 1$ во всех 14 (полный счёт цоколя); элементы порядка 2 из $\gamma_3 \cap M_4$ имеют ранг ровно на единицу меньше полного по модулю J_4 для 2^{84} (параметры `[2,4,6,1,2,-1,4,-2,-4,6,14]`), 2^{90}, 2^{107} и полный для остальных, при выполненной (X), так что эти числа точны: $D_4(Q) \neq 1$ **для** 2^{84}**,** 2^{90}**,** 2^{107} **сертифицировано**; $\alpha \in D_4$ для 2^{84} и 2^{90}, $\alpha \notin D_4$ для 2^{107}, 2^{79}, 2^{73} (`[2,4,6,1,1,1,1,1,1,6,14]`). Обе принадлежности есть и в трассах, а для 2^{90} трассу проигрывает ядро Lean (§3.2(f)). Точность неполных счётов проверяется принудительно (`socle-exact 1`). Для 2^{84}, 2^{90}, 2^{107} слова M_4 и M_5 — одни и те же 17, и проверяльщик точно (по Магнусу) показывает, что они лежат в $\gamma_3(F)$. Значит, некоторое их произведение $s \neq 1$ лежит в D_4: недостача одного по модулю J_4 даёт $s \in D_4$ по аддитивности (лемма 3, $4 \leq 2 \cdot 3$), а полный счёт по модулю J_5 даёт $s \notin D_5$. Поэтому $D_4(Q) \neq 1$ **держится только на копредставлении** (D5-21, пункт 6), без свойства базиса и без $\alpha \neq 1$. Копредставление здесь — обе срезки: записи K4 задают Q по модулю $\gamma_4(F)$, K5 — по модулю $\gamma_5(F)$ (487 и 1 505 соотношений, первые — подмножество вторых), а Q — 2-группа, так что в Δ/Δ^4 нет нечётного кручения и счёт по модулю 2^E есть счёт по модулю J_4. На доверии: построение Q в GAP и его pc-копредставление; для «$D_4 \cap \Omega_1(\gamma_3 \cap M_k) = 1$» в остальных группах — то, что тесты — базис $\Omega_1(\gamma_3 \cap M_k)$ (`IndependentGeneratorsOfAbelianGroup`, `JenningsSeries`); то, что $\alpha \neq 1$ там, где утверждается $\alpha \in D_4$. Вторая программа, пакет GAP lpres (независимый нильпотентный фактор, `cert/xcheck_lpres.py`), подтверждает и то и другое: 28 счётов цоколя из 33 записей типа Рипса — базисы $\Omega_1(\gamma_3 \cap M_k)$, и α имеет порядок 2 в 2^{84} и 2^{90} (`cert/README.md`, *Trusted inputs*).

Самодостаточная проверка в этом репозитории (`scripts/example.sh`): GAP сам выгружает SmallGroups порядков 64, 81 и 243, и вариант A, вариант B по перестановкам и вариант B по копредставлению сверяются между собой. Эталонные данные не нужны: 349 групп, 0 расхождений.

6. Рекомендации

6.1. Что запускать

- **Исходный перебор 3-групп порядка 3^7 больше не нужен**: все 3-группы порядка $\le 3^7$ класса 4 метабелевы, а для них $D_5 = \gamma_5$ по Гупте–Тахаре 1985. Для порядка 3^7 это уже сделано: B прошёл все 1202 кандидата за 157 с на трёх потоках, везде $D_n = \gamma_n$ при всех n. Вариант A на C независимо дал то же (4 905 с на трёх потоках): 1 202 группы, 6 010 уровней, 0 расхождений с B. Смысл ядра теперь — большие группы и 3-группы порядка 3^8 и выше.
- **Порядки 3^8, 2^{10} и выше.** Не экспортировать перестановки, а выгружать из GAP/ANUPQ pc-копредставление и только порождающие K (`export_pres.g` + `magnus_pres.py --kernel`). Стоимость на группу — построение J_5 ($M = 30..780$ при $d = 2..5$, доли секунды–секунды) плюс экспорт в GAP. Узкое место — порождение и сито в GAP, а не решётка. При $d \ge 6$ и $n = 5$ ($M \ge 1554$) выгоднее A на C по всей группе, пока $|G| \le 2^{11}$.
- **Группа Рипса (2^{38}).** Подходит B по копредставлению: группа класса 3, нужен $n = 4$, $M = \sum_{k<4} d^k \le 399$ даже при $d = 7$. Это то же, что уже посчитано для 3^{494} и для рипсовского семейства 2^{40}–2^{107} (секунды на группу). Не хватает копредставления самой группы Рипса: в открытых источниках оно не нашлось, нужна книга Михайлова–Пасси, LNM 1952. Поиск наименьших частных — это добавление одного соотношения и пересчёт J_4, тоже секунды.

6.2. Что делать с ядром дальше

1. Для B переписывать на C/FLINT имеет смысл только вместе с переходом к стандартным мономам (§3.6): на нынешних размерах ($M \le 780$) Python не узкое место.
2. A на C с одной группой на поток — запасной путь для $M \ge |G|$.
3. Точность: в каждом найденном «$\in$» нужен сертификат (`certified`) или HNF над $\mathbb{Z}$ (`--exact`).

6.3. Оценка для вопроса о плато [IMP26, §3.3]: $w \in D_7$?

$d = 6$, $n = 7$: $M = 6 + 6^2 + \cdots + 6^6 = 55\,986$. Тяжёлое вычисление не запускалось.

Ответ — в §6.5 (17.10.2026): $w \notin D_7(G)$, с сертификатом; плотный счёт на GPU занял 8 минут.

- **Плотно (как сейчас).** Одна форма Хауэлла $M \times M$ в `uint32` — 12.5 ГБ, с пулом строк вдвое больше. Число операций $\sim M^3/3 \approx 6 \cdot 10^{13}$ на один проход. На одном ядре (~10^9 операций/с в нашем C) — 15–20 часов. Но J_7 порождается не M, а $\sim \sum_r \sum_{t \le 6-\mathrm{ord}\, r} (t+1) 6^t$ строками: около $5 \cdot 10^4$ на соотношение с линейной частью, то есть порядка 10^6 строк, и замыкание требует нескольких таких проходов. Итого 10^{14}–10^{15} операций — дни на тестовой машине, и по памяти она на пределе (61 ГБ, общий с CI). **Не рекомендую.**
- **Стандартные мономы (§3.6).** Столбцов останется порядка $\log_2 |\Delta/\Delta^7|$ плюс мономы с необратимой опорой, то есть, вероятно, тысячи, а не $5.6 \cdot 10^4$. Это минуты. **Замер в §6.4: нет, не меньше 23 354 столбцов.** Нужна реализация (нильпотентный фактор ассоциативной алгебры над $\mathbb{Z}/2^E$) — несколько дней работы. Размер можно оценить заранее дёшево: посчитать $\log_2 |\Delta/\Delta^n|$ для $n = 2..5$ нынешним B ($M \le 1554$) и экстраполировать.
- **Без Γ, через G.** Если $G = M/K$ — конечная 2-группа, то её pc-копредставление с отсечением по весу < 7 уменьшает число соотношений, но не M. Поэтому от плотного варианта это не спасает.

6.4. Замер: размер базиса из стандартных мономов для группы плато

Оценка из §6.3 («тысячи столбцов») не подтвердилась. Ниже замер, а не прикидка:

- **Группа.** `gap/plateau.g` строит M и φ как в листинге 1 из [IMP26] и выписывает $G = M/K$ копредставлением на $x_1..x_6$: девять соотношений (3.12), $[[x_i, x_j], [x_k, x_l]]$, $c\, a^{-e_c}$ для левонормированных c веса 4 (это K) и коммутаторы веса 5 — всего 3 895 соотношений. Повторный запуск nq на этом копредставлении подтверждает его: класс 4, $G'' = 1$, $\gamma_4(G) = \langle a \rangle \cong C_{2^{14}}$, $w = a^{2^{13}} \ne 1$. Степени A^{-e_c} длинного слова A элемента a записаны как произведения степеней (`[[c,1],[A,-e]]`, их читает `magnus_pres.py`), иначе вход весит 340 МБ.
- **G бесконечна.** $G_{ab} = \mathbb{Z}^2 \oplus \mathbb{Z}/8 \oplus \mathbb{Z}/64 \oplus \mathbb{Z}/1024 \oplus \mathbb{Z}/16384$ (у x_4, x_6 нет степенных соотношений); ранги без кручения у γ_k/γ_{k+1} — $2, 1, 2, 0$. Значит, Δ/Δ^n бесконечна, $\log_2 |\Delta/\Delta^n|$ не определён и сертификат §3.2(d) неприменим; замена ему — ниже.
- **Правильная мера** числа столбцов — минимальное число образующих A_n^+/J_n, то есть $\dim_{\mathbb{F}_2} A_n^+/(J_n + 2A_n^+) = \dim_{\mathbb{F}_2} \Delta_{\mathbb{F}_2}/\Delta^n_{\mathbb{F}_2}$. В любом мономиальном базисе $\mathbb{Z}[G]/\Delta^n$, в том числе из стандартных мономов, мономов не

меньше. `magnus_pres.py` теперь выводит это число как `generators` (M_n минус ранг строк формы Хауэлла по модулю p; от E не зависит).

- **Дженнингс.** По теореме Дженнингса эта размерность — сумма коэффициентов при $t^1..t^{n-1}$ у $\prod_k (1+t^k)^{d_k}$, $d_k = \dim M_k/M_{k+1}$ (`py/jennings.py`). GAP даёт $G/P_7(G)$ порядка 2^{347} ($P_7 \subseteq M_7$, поэтому d_k при $k \le 6$ — те же, что у G): $d_k = 6, 21, 68, 22, 0, 68$; $d_5 = 0$, так как $\gamma_5(G) = 1$.

n	M_n (плотно)	`generators`, вариант B	Дженнингс	время B
2	6	6	6	
3	42	42	42	
4	258	256	256	5 с
5	1 554	1 226	1 226	31 с
6	9 330	5 492	5 492	820 с, 13 ГБ
7	55 986	—	**23 354**	—

Вариант B и счёт по Дженнингсу совпадают везде, где B запускался. Поле `generators` сверено с Дженнингсом и на всех SmallGroups порядков 64, 81 и 243 ($n \le 5$; `gap/export_jennings.g`, `scripts/compare_jennings.py`): 349 групп, 1 396 уровней, 0 расхождений. Время — у `magnus_pres.py` (numpy, одно ядро) на тестовой машине. Попутно B воспроизводит $w \in D_5, D_6$ и $a \notin D_5$ [IMP26, (3.15)]; для бесконечной G для этого нужен сертификат ниже.

Точность для бесконечной G. $A_n^+/J_n \cong \mathbb{Z}^r \oplus T$, где T — конечная 2-группа с инвариантами 2^{v_i}, поэтому `index_logp` по модулю 2^E равен $\sum_i \min(v_i, E) + Er$. Ранг r известен точно: по теореме Квиллена $\mathrm{gr}\,\mathbb{Q}[G] \cong U(\mathrm{gr}\,G \otimes \mathbb{Q})$, так что r_n — сумма коэффициентов при $t^1..t^{n-1}$ у $\prod_i (1-t^i)^{-h_i}$, где $h = (2, 1, 2, 0)$ — ранги без кручения: $r_n = 2, 6, 14, 27, 47, 78$ при $n = 2..7$; при $n \le 6$ это совпадает с числом столбцов без опоры в B. Если $\mathrm{index}(E_2) - \mathrm{index}(E_1) = r(E_2 - E_1)$, то все $v_i \le E_1$, и для элемента конечного порядка (его образ — кручение) принадлежность по модулю 2^{E_2}, $E_2 > E_1$, точна. Замер с `--E`:

n	r_n	index, $E = 20$	$E = 25$	$E = 30$	$\log_2\lvert T\rvert$
2	2	73	83	93	33
3	6	380	410	440	260
4	14	1 815	1 885	1 955	1 535
5	27	7 286	7 421	7 556	6 746
6	47	28 326	—	28 796	27 386

Разности ровно $5r_n$ (при $n = 6$ — $10r_6 = 470$), значит, экспонента T не больше 2^{20}, и принадлежности по модулю 2^{30} выше доказаны: $w \in D_6(G)$ и $a \notin D_5(G)$, как в [IMP26, (3.15)]. Тот же сертификат понадобится при $n = 7$ в плотном счёте.

[компьютер, сертифицировано] (D5-21, 01.11.2026; принадлежности есть и в трассах, §3.2(f)). Теперь эти ответы проверяет отдельная программа, `cert/check_in.c`, не делящая кода с вычислителями. Она сама строит $J_n + 2^E A_n^+$ по уровням (лемма N1) своим разреженным замыканием Хауэлла, проверяет, что результат замкнут относительно обоих сдвигов и обладает свойством Хауэлла, и прямо проверяет точность: столбцов без ведущего элемента ровно r_n, и $2^{E-1}e_c$ лежит в оболочке плюс эти столбцы для каждого c. Утверждения `results/cert/G_n{4,5,6}_E30.claim`, `G_n6_E32.claim`: $a \in D_4$, $w \in D_4, D_5, D_6$, $a \notin D_5, D_6$ и индексы таблицы (1 955, 7 556, 28 796; 28 890 при $E = 32$), каждое не дольше секунды на одном ядре (плотный B: 820 с и 13 ГБ при $n = 6$). Ранг A_n^+/J_n сертифицирован полиномиальными отображениями (§3.2(e), `results/cert/polymaps/G.json`); h и Квиллен больше не используются. $w \ne 1$ и $a \ne 1$ следуют из сертифицированных $w \notin D_7$, $a \notin D_5$ (§6.5). На доверии: копредставление и то, что порядки a и w — степени двойки (nq: $\gamma_4(G) = \langle a \rangle \cong C_{2^{14}}$, $w = a^{2^{13}}$): кручение нечётного порядка по модулю 2^E не видно (`cert/README.md`).

При $n = 7$ в любом мономиальном базисе не меньше **23 354** столбцов — 42 % от M. Реализация стандартных мономов сэкономила бы не больше чем в ~2.4 раза по столбцам, ~14 по кубу и ~6 по памяти; это несколько дней работы (нильпотентный фактор над $\mathbb{Z}/2^E$), и дни в минуты она не превращает. Граница достигается, только если все образующие можно взять мономами; сколько столбцов добавит кручение (порядки до 2^{14} есть уже в Δ/Δ^2), не замерялось.

Вывод. Для $d = 6$, $n = 7$ путь через стандартные мономы на CPU не даёт ожидавшихся «тысяч столбцов»; остаётся плотный счёт (на GPU или на CPU с ≥ 64 ГБ и днями). Для бесконечной G ответ «$\in$» по модулю 2^E точен при сертификате выше; ответ «$\notin$» точен, как и раньше.

6.5. GPU-ядро плотной формы Хауэлла по модулю 2^{32} (прототип)

`gpu/howell_gpu.py`; для бесплатного GPU — `gpu/howell_colab.ipynb` (Colab или Kaggle, T4 16 ГБ). При $p = 2$, $E \leq 32$ арифметика — переполнение uint32 (по модулю 2^E, $E < 32$, — то же и маска).

- **Алгоритм.** Форма Хауэлла из §2, блоками по $b = 256$ столбцов. В панели участвуют только строки, ненулевые в ней; шаги по столбцам — как в `c/hw.h` (опора наименьшей 2-адической оценки, нормировка к 2^v, точное деление, строка $2^{E-v}\cdot$опора в пул), и каждый шаг записывается коэффициентами G при опорных строках. Затем опорные строки на хвосте — $(I - SG_{\text{piv}})^{-1}SO$ (обращение унитреугольной матрицы удвоением), а прочие строки получают $O + G \cdot P$: одно произведение (строки $\times b$)·($b\times$ хвост) — основная работа.
- **Произведение по модулю 2^{32} точно.** Три способа, каждый при старте сверяется с точным целым произведением: `u32` (целочисленный matmul с переполнением), `f32` (байты во float32: при сдвиге s сумма $\sum_{i+j=s} a_i b_j$ по 64 столбцам $< 2^{24}$, точна; 10 произведений байтов, $i + j \leq 3$) и `i8` (байты со знаком на тензорных ядрах INT8 через `torch._int_mm`; аккумулятор int32 не переполняется) — схема по аналогии с Ozaki-II. На CPU работают первые два.
- **Сверка.** Две формы Хауэлла одного подмодуля сравниваются после приведения к приведённой форме Хауэлла, которая единственна: побайтно, по SHA-256. Случайные матрицы (все оценки, неполный ранг, E от 1 до 30, ширина панели 1–256; 1 500 случаев): расхождений с `c/hw.h` нет.
- **Группа плато** (вход `gpu/data/plateau_G.jsonl.gz`, вывод `gap/plateau.g`). Замыкание из `magnus_pres.J_howell` идёт поверх блочной формы: сдвиги подаются по два, остановка — «индекс не изменился» ($\text{span}\, H \subseteq \text{span}\, H'$ при равном индексе — равенство). Счёт по модулю 2^{32}; канонические формы образов по модулю 2^{30} и 2^{20} совпали с `magnus_pres.py` с `c/hw.h` при $E = 30, 20$, а по модулю 2^{32} — с `magnus_pres.py` с `py/howell.py` (`gpu/reference.json`), при $n = 4, 5, 6$. Числа §6.4 воспроизведены (`generators` 256, 1 226, 5 492; $a \notin D_5, D_6$, $w \in D_4, D_5, D_6$), свободных столбцов ровно r_n Квиллена (14, 27, 47), и $\text{index}(2^{32}) - \text{index}(2^{20}) = 12r_n$: сертификат выполнен.
- **Замер на CPU** (numpy-путь того же кода, на машине из §6.4): замыкание при $n = 6$ — 305 с, из них 269 с — произведения на хвосте ($8{,}4 \cdot 10^{11}$ умножений-сложений по модулю 2^{32}), шагов по столбцам 275 062; `magnus_pres.py` — 820 с. От $n = 5$ к $n = 6$ произведения растут как $M^{2{,}55}$.
- **На GPU** (Colab, A100-SXM4 40 ГБ, 16.10.2026, main 93a6259): `u32` точен, 11,2 Т умножений-сложений по модулю 2^{32} в секунду на $8192 \times 256 \times 8192$; `f32` точен, 0,67 Т/с; `i8` недоступен (`CUBLAS_STATUS_NOT_SUPPORTED`). Замыкание при $n = 4, 5, 6$: 2,3 / 17,6 / 127,9 с, все проверки PASS, канонические формы совпали с CPU. При $n = 6$ на произведения ушло 3,2 с из 128, остальное — шаги по столбцам, 0,88 мс на шаг (синхронизация с хостом и около двадцати малых ядер на столбец).
- **Для** $n = 7$ ($M = 55\,986$, $r_7 = 78$, то есть ожидается ранг 55 908):
  - *Строки соотношений.* `Words` строит $\mu(r) - 1$ сдвигами: $v \cdot \mu(x_g^e) = \sum_k C(e,k)\, v\, a_g^k$, а $v \cdot a_g$ переносит блок степени k в степень $k + 1$ одной индексной записью; произведения степеней слов по-прежнему через `Alg.mul`. На пуле процессов: $n = 6$ — 1,8 с вместо 54 с, строки те же побайтно; $n = 7$ — 11 с на 8 ядрах, 3 894 строки, 0,8 ГиБ.
  - *Память.* Форма хранится блоками (строки панели — только с её первого столбца: половина плотной $k \times M$, 5,8 ГиБ при $n = 7$). `howell()` принимает строки кусками (c_0, R) по возрастанию c_0 и кладёт кусок в пул только на панели, содержащей c_0; до неё его строки ни в одном шаге не участвуют, так что результат тот же (проверено: куски против целой матрицы, канонические формы совпали). Замыкание подаёт прежнюю форму блок за блоком и освобождает каждый блок по ходу; сдвиги берутся только от строк, ненулевых ниже старшей степени (около 9 300 строк при $n = 7$, 2,1 ГБ на матрицу сдвигов). Пик замыкания при двух матрицах сдвигов на вызов (замер на CPU через `tracemalloc`): 16,8 / 2,24 / 1,88 плотной матрицы M^2 uint32 при $n = 4, 5, 6$, доля падает с ростом M; при $n = 7$ это не больше $1{,}88 \cdot 11{,}7 \approx 22$ ГиБ [экстраполяция].
  - *Первый прогон $n = 7$ на A100 (16.10.2026, main 53658b5) упёрся в память*: раунды 0–3 за 3 / 26 / 57 / 85 с (ранг 3 962 → 20 975 → 33 142 → 38 258), пик пула 3,7 → 12,3 → 25,0 → 38,8 ГиБ, в раунде 4 не нашлось 3,4 ГБ. Замер на CPU не увидел двух вещей, заметных только при $n = 7$: сдвиги подавались целиком с нулевого столбца, и все их строки входили в пул на первой панели во всю ширину; обновление хвоста шло плитками по 16 384 строки — 3,7 ГБ на временный массив при $M = 55\,986$, и таких четыре (выбранные строки, произведение, сумма, маска), плюс три копии размером с пул при его пересборке.

Теперь: сдвиг — поток кусков по 2 048 строк B, каждый строится, когда `howell()` доходит до его первого столбца (сдвиги переводят столбцы монотонно, так что куски идут по возрастанию c_0; потоки сливаются по c_0) — сдвинутые строки начинаются в основном в старшей степени, поэтому входят поздно и уже; плитки по 2^{29} байт, временных массивов два; пул пересобирается в один новый буфер; кэш CuPy отдаётся драйверу, когда в нём больше 4 ГиБ свободного; шаг 8 берёт один сдвиг на вызов (`--group 1`). На CPU результаты те же ($n = 4, 5, 6$, все хэши), замыкание при $n = 6$ — 203 с вместо 368 с, пик — 1,64 плотной матрицы вместо 1,88.

– **Результат при** $n = 7$ (Modal, A100-SXM4 80 ГБ, 16.10.2026, main 9114cb9, `gpu/modal_run.py`; шаги по столбцам `cuda`, один сдвиг на вызов). Сначала проходят `selftest`, `random --big` (все три режима шагов) и $n = 4, 5, 6$ против `gpu/reference.json`. Замыкание: раунды 0–6 за 2 / 31 / 52 / 69 / 100 / 117 / 118 с (ранг 3 962 → 20 975 → 33 142 → 38 258 → 51 072 → 55 908 → 55 908; индекс 1 666 980 → … → 108 940 → 108 940), всего 488 с, 73 вызова Howell, $5{,}1 \cdot 10^{14}$ умножений-сложений по модулю 2^{32} за 227 с, 3,75 млн шагов по столбцам; пик пула 16,4 ГиБ (в раунде 3 — 10,2 ГиБ против прежних 38,8); форма 5,86 ГиБ. Сводка, канонические формы и образы по модулю $2^{30}, 2^{20}$ — 138 с.

E	индекс	ранг	каноническая форма, sha256
32	108 940	55 908	`e060b5e742633ba9…`
30	108 784	55 908	`464917aa22bfe1dd…`
20	108 004	55 908	`905de7e5675eb7b7…`

Свободных столбцов $78 = r_7$; $\mathrm{index}(2^{32}) - \mathrm{index}(2^{30}) = 156 = 2r_7$ и $\mathrm{index}(2^{32}) - \mathrm{index}(2^{20}) = 936 = 12r_7$, так что кручение имеет экспоненту $\le 2^{20}$ и принадлежности точны: $a \notin D_7(G)$, $w \notin D_7(G)$; вместе с §6.4 $w \in D_6(G) \setminus D_7(G)$. Ранг по модулю 2 даёт 23 354 образующих — ровно нижнюю оценку §6.4 для мономиального базиса. [CPU-эталона при $n = 7$ нет: результат проверяют только сертификат и сверка с r_7.] Форма по модулю 2^{32} лежит в томе Modal `cflint-runs`, `20261016-195219_9114cb9/J7_E32.npz`. **[компьютер, сертифицировано]** (D5-21, 30.10.2026). Теперь у обеих непринадлежностей есть сертификаты, которые проверяет отдельная программа: `results/cert/G_n7_{a,w}.cert.gz`, функционал $\lambda : A_7^+ \to \mathbb{Z}/2^{32}$ с $\lambda(J_7) = 0$ и $\lambda(\mu(w) - 1) \neq 0$. `cert/check_notin.c` не делит кода с вычислителями. Она сама строит $\mu(r) - 1$ по копредставлению и проверяет $\lambda(u(\mu(r) - 1)v) = 0$ для всех мономов u, v. На одном ядре это 6,5 с на сертификат и 52 МБ. Для непринадлежности условие точности по E не нужно. На доверии остаются копредставление, записанное `gap/plateau.g` (как копредставление G по модулю γ_7: соотношения из $\gamma_7(F)$ в нём опущены), теорема §3.1 и компилятор C (`cert/README.md`). Индекс и оба ответа воспроизводит и `cert/check_in.c` (01.11.2026; `results/cert/G_n7_E32.claim`): индекс 108 940 и 55 908 строк при $E = 32$, 78 свободных столбцов $= r_7$, за 3 с на одном ядре.

– *Шаги по столбцам.* Три реализации одной арифметики в одном порядке: `host` (как было, синхронизация на каждом столбце), `dev` (скаляры на устройстве, без синхронизации; обратный к единице — итерацией Ньютона), `cuda` (три ядра на столбец: редукция для опоры — наименьшая оценка, затем первая строка; опорная строка со столбцом множителей; обновление ранга один). На CPU `host` и `dev` совпали с `c/hw.h` на всех случайных матрицах и с `gpu/reference.json` при $n = 4, 5$ (`dev`) и $n = 4, 5, 6$ (`host`). `cuda` проверяется в блокноте (шаг 3) до использования.

– *Каноническая форма и принадлежность* — блоками, на устройстве. Форму можно сохранить (`--save`).

– Шаг 8 блокнота считает $n = 7$ на GPU не меньше 38 ГБ. CPU-эталона при $n = 7$ нет: проверки — свободные столбцы против r_7 и сертификат $\mathrm{index}(2^{32}) - \mathrm{index}(2^{30}) = 2r_7$, $\mathrm{index}(2^{32}) - \mathrm{index}(2^{20}) = 12r_7$.

6.6. D5-7: поиск метабелевой группы класса 4 с $D_7 \neq 1$ (первый проход)

Подробности, таблица семейств и числа — в английской версии, `docs/analysis.md` §6.6; сырые строки — в `results/c4/`.

- **Сведение** [на бумаге]: достаточно коциклических факторов $G_\varphi = M/\ker\varphi$, $M = \Gamma/\gamma_5\Gamma''$, $\varphi : \gamma_4(M) \twoheadrightarrow C_{2^m}$, и в каждом — одного элемента $A^{2^{m-1}}$ порядка 2 в $\gamma_4(G_\varphi)$. Ответ «$\notin$» по модулю 2^{32} точен без сертификата.
- **Конвейер**: GAP+nq в Docker (`gap/docker`, `scripts/gap.sh`) → `gap/c4search.g` (семейства `gap/c4_plateau_quot.g`, `gap/c4_lie1.g`, `gap/c4_rand.g`) → `gpu/howell_gpu.py batch` (CPU) или `gpu/modal_batch.py` (Modal A10G: $d = 6$, $n = 7$ — 8–9 мин на группу). Проверка: группа плато, пересобранная заново, даёт индексы 1983, 7610, 28 890 и 108 940 при $n = 4..7$, как раньше.

- **[компьютер, сертифицировано]** (D5-21, 02.11.2026; принадлежности есть и в трассах, §3.2(f)). Для 98 групп G_φ, чьи записи лежат в репозитории (`gpu/data/c4/`: L, P6sel, R, R0, R2, S), `cert/check_in.c` подтверждает сохранённые ответы (107 пар «группа, уровень»: индекс по модулю 2^{32} и принадлежность каждого теста, нетривиального в G_φ, включая $w \in D_6$ для R0 и $A^{32} \in D_5 \setminus D_6$ для L1, L1b128, L1b256), а для 16 выживших R, R2 — ещё и $w \in D_6$ (их сохранённые прогоны только при $n = 7$; ещё 16 утверждений), с проверкой точности (X): `results/cert/c66/`, 123 утверждения, записи по одной на группу в `results/c4/groups/`. 746 групп T в репозиторий не попадали, их φ выбирались случайно, но каждая есть $M/\ker\varphi$ с $\ker\varphi \subseteq \gamma_4(M)$, так что §6.8 даёт $D_5(G_\varphi) = 1$ для всех них; для P6, R, R2 он даёт больше проверенных здесь ответов «$\notin D_7$»: $D_7(G_\varphi) = 1$. На доверии: записи (GAP + nq); для T — что её базы суть 40 баз, перестроенных в §6.8 по `gap_T_in.log`.
- **Итог**: ни в одной из проверенных групп $D_7 \cap \gamma_4 \neq 1$ нет. $D_6 \cap \gamma_4 \neq 1$ — только у самого плато и тех его факторов, где w выживает. Плато жёсткое: любое изменение одного показателя в (3.12) и любое отождествление x_5 убивают w уже в M. Групповой вариант L_1 теряет степень ($D_5 \neq 1$, $D_6 \cap \gamma_4 = 1$).

6.7. Класс 5 (D5-7): сдвиг плато [IMP26, §3.3]

Вопрос: есть ли метабелева группа класса 5 с $D_8 \neq 1$ (групповой аналог кольца L_2 из теоремы A [IMP26]), и тогда $D_9 \neq 1$? Плотный тест D_9 посилен только при $d \leq 4$ образующих ($M = 87\,380$; при $d = 5$ — 488 280).

Инструменты. `gap/plateau5_lib.g` по копредставлению Γ на $x_1..x_d$ строит $M = \Gamma/\gamma_{c+1}\Gamma''$ (nq), проекцию φ группы $\gamma_c(M)$ на один циклический множитель её базиса Смита, на котором $w \neq 1$, и $G = M/\ker\varphi$ в виде копредставления (соотношения; $[U, V]$ для левонормированных U, V весов ≥ 2 с суммой $\leq c$; $C\,c_0^{-e_C}$ для левонормированных C веса c; коммутаторы веса $c + 1$) в JSON для `gpu/howell_gpu.py`. Строение G (класс, $G'' = 1$, γ_k/γ_{k+1}, ранги без кручения для сертификата Квиллена, $|w|$) читается с pcp-фактора $M/\ker\varphi$; повторный nq на записанном копредставлении (как в `gap/plateau.g`) — по желанию: на этих показателях nq переполняется (`CombiCollect`). Семейства — `gap/plateau5_cands.g`, прогон GAP и замыкания — `scripts/plateau5_scan.py`, то же на Modal — `gpu/modal_c5.py` (GAP 4.14 + nq на CPU-контейнере, замыкание на A100 80 ГБ), GAP в Docker локально — `scripts/gap5.sh`. `howell_gpu.py plateau` теперь принимает любой вход и любое число тестов (`test_names`); с CPU-эталоном сверяется только вход по умолчанию. `random` и `plateau --n 4 5 6` на CPU по-прежнему проходят против `gpu/reference.json`.

Сверка на [IMP26, §3.3]. Библиотека с φ, выбранной по a (листинг 1), даёт $w \in D_6$, $a \notin D_6$ (сертифицировано), как в §6.4–6.5.

Какие соотношения (3.12) нужны ($n = 6$, всё сертифицировано). Без четвёртого соотношения $x_5^{1024} d_2^{16}$ теряется $w \in D_6$; без любого из пяти остальных (d_5^{2048}, d_1^{16}, d_2^{128} и двух верхних) — нет, и без всех пяти сразу тоже. Совсем без x_5 $w \notin D_6$; без d_5^{16} во втором соотношении $w = 1$. Свободный образующий x_6 можно заменить на x_3 ($t = [x_4, x_3]$): пять образующих, $w \in D_6$. Итак, плато класса 4 живёт на пяти образующих с четырьмя соотношениями:

$$x_1^8 d_2 d_3^4, \quad x_2^{64} d_1^{-1} d_3^{-16} d_5^{16}, \quad x_3^{16384} d_1^{-4} d_2^{16}, \quad x_5^{1024} d_2^{16}, \qquad d_i = [x_4, x_3, x_i].$$

Четыре образующих, класс 4: во всех попытках $w \notin D_6$ или $w = 1$. Группа типа Рипса из [BM23, §1.2] с z, заменённым на $t = [z, e_2]$, и её масштабирования: $w \in D_5 \setminus D_6$ (и для её жадного максимального фактора); (3.12) без x_5, $t = [x_4, x_3]$: $w \in D_5 \setminus D_6$; x_5, слитый с образующим t, или $x_5 := W(x_1..x_4)$ для восьми слов W, или t только из образующих с кручением (десять вариантов): $w = 1$ в M.

Класс 5. $t = [x_4, x_3, x_3]$, $d_i = [t, x_i] \in \gamma_4$, те же соотношения, $c = 5$, $\gamma_5(G) \cong C_{2^{14}}$:

группа	d	D_6	D_7	D_8
типа [BM23], $t = [z, e_2, e_2]$ (три масштаба)	4	$w \in$	$w \notin$	
(3.12) без x_5	4	$w \in$	$w \notin$	
(3.12) со сдвигом, пять образующих (`plateau5_c5i5`)	5	$w \in$	$w \in$	$w \notin$
то же с четырьмя соотношениями; с удвоенными L_{13}, L_{23}; с $E_5 = 512$	5	$w \in$	$w \in$	—

Всё сертифицировано (свободные столбцы $= r_n$, разности индексов $2r_n$ и $12r_n$). Для `plateau5_c5i5` при $n = 8$: $M = 97\,655$, замыкание за 7 раундов и 1 829 с на A100 80 ГБ (пик 58,8 ГиБ), ранг 97 648, свободных столбцов $7 = r_8$, индекс 169 278, $\text{index}(2^{32}) - \text{index}(2^{30}) = 14$, $-\text{index}(2^{20}) = 84$. Так как $\gamma_5(G)$ циклическая и w — её элемент порядка 2, из $w \notin D_8$ следует $D_8 \cap \gamma_5 = 1$; для варианта с четырьмя соотношениями тесты $u_3^{2^j}$ дают $D_6 \cap \gamma_5 = \langle w \rangle$. Итак, у этой группы $D_7 \neq 1$ ($f_{\text{met}}(5) \geq 7$), но не $D_8 \neq 1$.

Почему сдвиг останавливается на D_7 (лемма о спаривании). Если $x^E \in \gamma_k$, $E = 2^s \geq 4$, то $E(x-1) \in \Delta^2$, $2E(x-1) \in \Delta^3$, $4E(x-1) \in \Delta^4$ (разложить $x^E - 1 = \sum_m \binom{E}{m}(x-1)^m$; на последнем шаге $4\binom{E}{3} = \frac{(E-1)(E-2)}{3} \cdot 2E$ и $3 \mid (E-1)(E-2)$). Отсюда $(x-1)(\zeta^{2^{k-2}E} - 1) \in \Delta^{2k}$ при $\zeta \in \gamma_k$, $k \leq 4$: это множители $x_1^{16} = x_1^{2E_1}$, $x_2^{128} = x_2^{2E_2}$ в (3.14) ($k = 3$). Для класса 5 ($k = 4$) аналог требует $g_i = \prod_j x_j^{L_{ij}} = \zeta_i^{4E_i}$, а в (3.12) $g_1 = \zeta_1^{16}$, $g_2 = \zeta_2^{128}$, то есть $2E_i$. `scripts/plateau5_pairing.py` перебирает масштабирования степенями двойки E_i, строк матрицы показателей и L_{ij}, при которых $g_i \in 4E_iD$ по модулю γ_c (121 644 варианта); у 80 самых дешёвых, построенных в классе 4, везде $w = 1$ в M. В этом семействе два требования тянут в разные стороны: для $w \neq 1$ нужно $L_{ij} < E_iE_j$ хотя бы для одной пары (множители вида $[x_j, x_i^{E_i}]$ имеют порядок, делящий E_j), а учетверённое спаривание поднимает L_{ij}.

Кольца Ли на четырёх образующих (на бумаге, по доказательствам [IMP26, §3.1–3.2]). В L_N свободный образующий x_5 можно заменить на x_3: на $x_1..x_4$ с порядком $x_3 < x_1 < x_2 < x_4$, $t = [x_4, x_3, \dots, x_3]$ (N букв x_3), с теми же r_i и (ii)–(iv), получается $D_{N+4} = \cdots = D_{2N+4} = \langle a \rangle \cong \mathbb{Z}/2$, $D_{2N+5} = 0$. Меняется вот что: в лемме о верхнем слое скобка $e_3[x_3, \dots]$ веса $N+3$ теперь может попасть в u_1, u_2, но $64\mu(u_1)$, $64\mu(u_2) \equiv 0 \bmod 256$; в лемме о последней начальной форме c_i-координата скобки с r_3 кратна $e_3 = 64$, а на неё делятся e_1, e_2. При $N = 2$: класс 5, $D_6 = D_7 = D_8 \neq 0 = D_9$ на четырёх образующих. Именно групповой аналог этого кольца таблица выше построить не смогла.

Как повторить.

```
# группа класса 5 (пять образующих); около 9 мин, Docker, образ из gap/Dockerfile.plateau5
scripts/gap5.sh plateau5_cands.g 'P5NAME := "c5i5"; P5I5 := rec(cls := 5); P5OUT := "/data/c5i5.jsonl";'
# D_n на GPU с памятью от 64 ГБ для n=8 (закоммиченный plateau5_c5i5.jsonl.gz — этот файл)
python3 gpu/howell_gpu.py plateau --input gpu/data/plateau5/plateau5_c5i5.jsonl.gz --n 6 7 8 --h 1 0 0 0 0 --
# на Modal (в фоне; результаты в томе math43-c5)
modal deploy gpu/modal_c5.py
python gpu/modal_c5.py cpu|gpu '<команда>' ...
# перебор кандидатов (SCAN_DEVICE=gpu; SCAN_REUSE=1 — только замыкание)
python3 scripts/plateau5_scan.py LOG 'name|I5rec(cls:=4, ...)|5,6'
```

Расход Modal на весь §6.7 — около 8 долларов. Жадный максимальный фактор группы класса 5 с сохранением w (`P5_Quot`, 65 баз) за 1 ч 45 мин GAP не досчитался и был остановлен.

6.8. D5-7, класс 4: пробел $D_7 \not\subseteq \gamma_4$ и все $K \subseteq \gamma_4(M)$ сразу

Подробности и доказательства — в английской версии, `docs/analysis.md` §6.8; строки — `results/c4/q3/`, `results/c4/kfree/`.

(а) Фактор ступени 3.

- **Лемма 1** [на бумаге]. Q конечно порождена, нильпотентна ступени ≤ 3, $n \in \{4, 5\}$. Тогда $D_n(Q) \subseteq \tau(\gamma_3(Q))$ ($D_3 = \gamma_3$ [MP09] †; рациональные размерные подгруппы — изоляторы, Jennings †), и $g \mapsto g-1$ — гомоморфизм $\gamma_3(Q) \to \Delta^3/\Delta^n$ (так как $(g-1)(h-1) \in \Delta^6$). Значит, при 2-группе $\tau(\gamma_3(Q))$: $D_n(Q) = 1 \iff$ этот гомоморфизм инъективен на элементах порядка 2.
- **Следствие 2** [на бумаге]. Если $\gamma_5(G) = 1$ и $D_5(G/\gamma_4) = 1$, то из $D_r(G) \cap \gamma_4 = 1$ следует $D_r(G) = 1$ ($r \geq 5$).
- $G_\varphi/\gamma_4 = \Gamma/\gamma_4(\Gamma)\Gamma''$ не зависит от φ: 80 групп Q покрывают все 844 группы §6.6.
- **Счёт** [компьютер]: во всех 80 $\tau(\gamma_3(Q))$ — 2-группа, и инъективность есть уже при $n = 4$, т. е. $D_4(Q) = 1$. Инъективность по модулю 2^{32} точна без сертификата (сертификаты всё равно выполнены).
- **Итог (а)** [компьютер + на бумаге]: группа плато G и три группы P6 — $D_7 = 1$ (для G теперь без опоры на описание $D_6(G)$ из [IMP26]); 16 выживших факторов R, R2 — $D_7 = 1$; 62 группы L — $D_6 = 1$ (54 из них — $D_5 = 1$); 16 групп S и 746 групп T — $D_5 = 1$. Это утверждения о проверенных коциклических G_φ, а не о M.
- **Связь с D5-2** [на бумаге]: элемент $D_7(G) \setminus \gamma_4$ дал бы $D_5 \neq 1$ в конечной 2-группе ступени 3 (финитная аппроксимируемость конечными 2-группами на 2-элементах, Gruenberg †) — отрицательный ответ на D5-2. Закрытый здесь пробел — ровно частные случаи D5-2; среди 80 групп три конечные (Rf15, Rf16, Rd46f15), у всех $D_4 = 1$. Это примеры, а не общий аргумент.

(б) Все K сразу. $M = \Gamma/\gamma_5\Gamma''$, $A_4 = \gamma_4(M)$, $\psi(k) = k - 1$, $n \leq 8$.

- **Лемма 3** [на бумаге]. $\psi : A_4 \to \Delta^4/\Delta^n$ аддитивно; $v \in D_n(M/K) \iff v - 1 \in \Delta^n + \sum_k \psi(k)R$, R — константа и мономы $(x_{j_1} - 1)\cdots(x_{j_l} - 1)$, $1 \leq l \leq n-5$, k по образующим K. При фиксированном K — линейная задача.

- **Где ломается линейность** [на бумаге]. «Существует K» — это объединение по подгруппам: нужна точка аффинного смежного класса $\Lambda^{-1}(\psi(w)) \subseteq A_4^R$, координаты которой порождают подгруппу без w. Двойственно — условие ранга один на тензор в $\widehat{A_4} \otimes \widehat{R}$; линейным оно не является. Одной формой Хауэлла ответ «да» не получить.
- **Предложение 4** [на бумаге] — то, что остаётся линейным. $U_n = J_n + \Lambda(A_4 \otimes R_+)$. Если $\bar{\psi} : A_4 \to A/U_n$ инъективно, то $D_n(M/K) \cap \gamma_4 = 1$ для **всех** $K \subseteq \gamma_4(M)$; вместе с (а) — $D_n(M/K) = 1$. Ядро $\bar{\psi}$ содержит все кандидаты (фильтр). Ядро — кручение (по Квиллену), поэтому достаточно элементов порядка 2; инъективность по модулю 2^{32} точна; из инъективности при n следует она же при больших n.
- **Реализация**: `gap/c4_q3.g` (`Q3_C=4`) → `scripts/c4_q3.py prep --kgens` (решение по модулю 2^{32}, проверка по точным целым) → `howell_gpu.py batch --kfree`.
- **Результаты** [компьютер]:
  - T (40 баз, $d = 4$): $\bar{\psi}_5$ инъективно во всех — $D_5(M/K) = 1$ для всех K;
  - L: $\bar{\psi}_6$ инъективно у всех 7 (у L1x2, L1x4 уже $\bar{\psi}_5$) — $D_6(M/K) = 1$ для всех K;
  - S (16 баз): $\bar{\psi}_5$ инъективно — $D_5(M/K) = 1$ для всех K;
  - **плато P6** ($M = \Gamma/\gamma_5\Gamma''$ для (3.12), $d = 6$, $\gamma_4(M) = \mathbb{Z}^3 \oplus$ 2-группа, 205 элементов порядка 2): при $n = 5, 6$ (CPU) ядро $\bar{\psi}$ — ровно $\langle w \rangle$, и $D_5(M) \cap \gamma_4 = D_6(M) \cap \gamma_4 = \langle w \rangle$; при $n = 7$ (Modal A10G, 890 с, пик 20,2 ГиБ; сертификат J_7: свободных столбцов $99 = r_7$ при $h = (2, 1, 2, 3)$, разности индексов $198 = 2r_7$, $1188 = 12r_7$) $\bar{\psi}_7$ **инъективно** (205 из 205).
- **Итог (б)** [компьютер + на бумаге]: для группы плато $D_7(M/K) = 1$ для **любой** $K \subseteq \gamma_4(M)$ — то есть для всех метабелевых групп ступени 4 вида Γ/N, $\gamma_5\Gamma'' \subseteq N \subseteq \gamma_4(\Gamma)\gamma_5\Gamma''$, включая все φ, которые §6.6 не перебирал. Для баз T, S — $D_5(M/K) = 1$, для L — $D_6(M/K) = 1$ при всех K: перебор φ в §6.6 для них оказался полным.
- **Факторы плато по соотношениям вне** γ_4 (R, R2; у каждого своё M) [компьютер]: при $n = 6$ у всех 11 построенных баз ядро ранга ровно 1 (место w). При $n = 7$ прогнана одна база, **Rd46** ($[x_4, x_6, x_4] = [x_4, x_6, x_6] = 1$): A10G, 885 с; сертификат J_7 ($49 = r_7$ при $h = (2, 1, 0, 0)$, $98 = 2r_7$, $588 = 12r_7$); $\bar{\psi}_7$ инъективно (197 из 197) — $D_7(M/K) = 1$ при всех K.
- **Остальные 10 баз R2 при** $n = 7$ [компьютер]. Та же процедура: M строит `gap/c4_q3.g` (`Q3_C=4`) на CPU-контейнере Modal (8 ядер, 4 параллельных GAP, 9,6–10,1 мин на базу; логи `results/c4/kfree/gap_m4_R2n7{a,b,c,d}.log`), затем по одному прогону A10G на базу (`gpu/modal_batch.py spawn --ref 8ed65af ... -- --n 7 --kfree --group 1`, 804–845 с; строки `results/c4/kfree/m4_<база>_n7.out.jsonl`). У всех десяти $h = (2, 1, 2, 3)$, $r_7 = 99$, $\dim A_7 = 55\,986$; сертификат J_7 выполнен: свободных столбцов $99 = r_7$, разности индексов $198 = 2r_7$ и $1188 = 12r_7$; ни один элемент порядка 2 из $\tau(\gamma_4(M))$ не лежит в J_7; $\bar{\psi}_7$ **инъективно**:

база	соотношение	образующих $\gamma_4(M)$	эл-тов порядка 2	индекс J_7 ($2^{32}/2^{30}/2^{20}$)	индекс U_7	ранг по модулю U_7
Rk14	$[x_1, x_4]$	186	183	137 835 / 137 637 / 136 647	101 781	183 из 183
Rk15	$[x_1, x_5]$	186	183	137 907 / 137 709 / 136 719	101 786	183 из 183
Rk16	$[x_1, x_6]$	186	183	137 835 / 137 637 / 136 647	101 781	183 из 183
Rk24	$[x_2, x_4]$	186	183	132 116 / 131 918 / 130 928	97 590	183 из 183
Rk26	$[x_2, x_6]$	186	183	132 116 / 131 918 / 130 928	97 590	183 из 183
Rk34	$[x_3, x_4]$	187	184	127 666 / 127 468 / 126 478	94 254	184 из 184
Rk35	$[x_3, x_5]$	187	184	128 896 / 128 698 / 127 708	95 148	184 из 184
Rk36	$[x_3, x_6]$	187	184	127 666 / 127 468 / 126 478	94 254	184 из 184
Rk45	$[x_4, x_5]$	187	184	128 879 / 128 681 / 127 691	95 147	184 из 184
Rk56	$[x_5, x_6]$	187	184	128 879 / 128 681 / 127 691	95 147	184 из 184

  Пары Rk14/Rk16, Rk24/Rk26, Rk34/Rk36, Rk45/Rk56 дают одинаковые числа (видимо, симметрия $x_4 \leftrightarrow x_6$ в (3.12); не проверялось). При $n = 6$ у каждой было ядро ранга 1, при $n = 7$ оно исчезает — как у плато. С (а) и Предложением 4: $D_7(M/K) = 1$ **для всех** $K \subseteq \gamma_4(M)$ для каждой из 10 баз.
- **Последние 5 баз — R (Rd4, Rd6, Rf15, Rf16) и Rd46f15 — при** $n = 7$ [компьютер]. Та же процедура: M строит `gap/c4_q3.g` (`Q3_C=4`, `Q3_NQTOP := false`) на CPU-контейнере Modal (5 ядер / 28 ГиБ, пять GAP параллельно с лимитом 1900 с, `scripts/c4_rest_gap.sh`; логи `results/c4/kfree/gap_m4_<база>.log`; приложение `math43-c4rest-gap`), 10–15 мин на базу; затем два вызова A10G (`gpu/modal_batch.py spawn --ref efecbc5`

`... -- --n 7 --kfree --group 1`, приложение `math43-c4rest`), 820–901 с на базу, пик GPU 20,2–20,9 ГиБ; строки `results/c4/kfree/m4_<база>_n7.out.jsonl`. Везде $\dim A_7 = 55\,986$, ни один элемент порядка 2 из $\tau(\gamma_4(M))$ не лежит в J_7, и $\bar{\psi}_7$ **инъективно**:

база	соотношения к (3.12)	M	h	образующих $\gamma_4(M)$	эл-тов порядка 2	индекс J_7 ($2^{32}/2^{30}/2^{2}$
Rd4	$[x_4, x_6, x_4]$	бесконечна	$(2,1,1,1)$	202	201	145 421 / 145 281 / 1
Rd6	$[x_4, x_6, x_6]$	бесконечна	$(2,1,1,1)$	202	201	145 421 / 145 281 / 1
Rf15	$x_4^{2^{15}}, x_6^{2^{15}}$	конечна	0	208	208	147 440 / 147 440 / 1
Rf16	$x_4^{2^{16}}, x_6^{2^{16}}$	конечна	0	208	208	147 662 / 147 662 / 1
Rd46f15	$[x_4, x_6, x_4], [x_4, x_6, x_6], x_4^{2^{15}}, x_6^{2^{15}}$	конечна	0	197	197	141 071 / 141 071 / 1

Сертификат Rf16 не выполнен, как и в §6.6 (A/J_7 имеет элементы порядка $> 2^{20}$), но здесь он не нужен: все ответы — «$\notin$»/инъективность, а они точны по модулю 2^{32} без сертификата. Rd4 и Rd6 дают одинаковые числа (видимо, $x_4 \leftrightarrow x_6$). С (a) и Предложением 4: $D_7(M/K) = 1$ **для всех** $K \subseteq \gamma_4(M)$ для всех пяти.

- **Почему nq «шёл часами»** [компьютер]: не воспроизвелось. `gap/c4_q3.g` теперь печатает время этапов. На Modal (GAP 4.14, Debian trixie), без передачи в nq коммутаторов веса 5 (`Q3_NQTOP := false`: при `class := 4` они избыточны, в выходных соотношениях остаются): nq — 4 с для бесконечных Rd4, Rd6 и 132 с (Rf15, Rd46f15), 267 с (Rf16) для конечных: большие показатели $2^{15}, 2^{16}$ дороже в 30–60 раз, но это минуты. Дальше у всех одинаково: нижний центральный ряд ~1 с, инварианты и snf-базис γ_4 30–43 с, показатели 540 коммутаторов 8 с, а затем ~570 с (90 % процессорного времени GAP) — проверка, что дополнительное тестовое слово w лежит в $\gamma_4(M)$ и нетривиально (`MappedWord` длинного слова и принадлежность γ_4). Именно это, а не nq, даёт 10 мин на базу (видимо, и для R2 — там времён этапов в логах нет). Были ли часы в Docker из-за 3240 лишних соотношений для конечной группы или из-за памяти Docker VM — не проверялось.
- **Итог первого прохода класса 4 для «всех** K**»** [компьютер + на бумаге]: тест проведён при решающем n для **всех** 80 баз §6.6: P6, 12 баз R2 (Rk14…Rk56, Rd46, Rd46f15) и 4 базы R (Rd4, Rd6, Rf15, Rf16) — $D_7(M/K) = 1$ (17 баз); L — $D_6(M/K) = 1$ (7); T и S — $D_5(M/K) = 1$ (56), при любой $K \subseteq \gamma_4(M)$. Ни одна метабелева группа ступени 4, получаемая из этих копредставлений центральной модификацией γ_4, не имеет $D_7 \neq 1$; для контрпримера нужна база Г вне этого семейства.
- **Дальше**: (1) сделано для всех баз первого прохода (выше); (2) при неинъективном $\bar{\psi}_n$ ядро — список кандидатов; для $v \in \ker \bar{\psi}_n$ представление $\psi(v) = \sum \psi(k_j) m_j$ по модулю J_n с $v \notin \langle k_j \rangle$ даёт контрпример $M/\langle k_j \rangle$ — нужны коэффициенты (расширенная форма Хауэлла), не реализовано; (3) тот же тест для класса 5 (группа §6.7: ψ аддитивно по модулю Δ^{10}).
- **[компьютер, сертифицировано]** (D5-21, 01.11.2026; принадлежности есть и в трассах, §3.2(f)). Все индексы J_n и U_n и все числа битов цоколя этого раздела (по модулю 2^{32}) подтверждает `cert/check_in.c`, не делящий кода с `howell_gpu.py` (260 утверждений в `results/cert/c4/`, `cert/README.md`): (a) для всех 80 групп Q при $n = 4, 5$ элементы порядка 2 из $\tau(\gamma_3(Q))$ независимы по модулю $J_n + 2^{32} A_n^+$ (значит, θ инъективно и $D_4(Q) = 1$); (b) для M каждой базы — index(J_n), index(U_n) и число битов цоколя, как в таблицах: T, S при $n = 5$ и L при $n = 5, 6$ (включая неинъективные L1, L1b128, L1b256 при $n = 5$ и ранг 1 из 2 для L1free, L1x2free), P6 при $n = 5, 6$ (204 из 205, $w \in J_5, J_6$) и $n = 7$ (205 из 205), Rk14, …, Rk56 и Rd46 при $n = 6$ (ранг на единицу меньше полного), а также Rk14, …, Rk56, Rd46, Rd46f15, Rd4, Rd6, Rf15, Rf16 при $n = 7$ (**инъективно**); каждая база при $n = 7$ — около 15 с на одном ядре (A10G: 800–900 с). Всё вместе на kimmy — 13 мин (802 с, одно ядро; tm `cert-check-c4`, 20261101-182226). Записи GAP в прогонах выше не сохранялись: 32 восстановлены с томов Modal (31 группа M и Q для P6), остальные 128 построены заново (`scripts/c4_rebuild_gap.sh`; T — по параметрам из журнала `gap/c4_rand.g`, `gap/c4_T_fromlog.g`), и с ними проверяльщик воспроизводит все индексы и числа исходных выводов; для перестроенных записей утверждаются только эти числа, не зависящие от базиса, для сохранённых — и принадлежность каждого теста (кроме Rk14, …, Rk56 при $n = 6$: тот прогон шёл по более ранней записи без w). Не перепроверены: индексы по модулю 2^{30}, 2^{20}. На доверии: записи GAP (GAP 4.14 + nq; все соотношения; `test_coeffs` — это элементы $(\mathrm{ord}_t/2)\, b_t$ snf-базиса b_t группы γ_c, все они, а `kgen_coeffs` — этот базис; первое сверяется со вторым там, где γ_4 конечна, — в 51 из 80 групп M), лемма 3 и на бумаге лемма 1, предложение 4, следствие 2. Вторая программа lpres (`cert/xcheck_lpres.py`) подтверждает базис цоколя для 77 из 80 записей класса 3 и 47 из 80 записей класса 4, то, что `kgen_coeffs` порождают γ_4, для этих 47, и 2-степенные порядки членов для 5 из них с утверждениями о принадлежности (L1, L1b128, L1b256, L1free, L1x2free); остальные не уложились в 30 мин.

- **Расход Modal на §6.8**: \$0,79 (CPU-контейнер для GAP — \$0,07; два прогона A10G при $n = 7$ — \$0,72), приложение `math43-kfree` остановлено. 10 баз R2 при $n = 7$: \$4,03 (CPU-контейнер для GAP, 8 ядер / 48 ГиБ, 35 мин — \$0,44, `math43-r2n7-gap`; десять прогонов A10G — \$3,59, `math43-r2n7`, остановлено). Последние 5 баз (R, Rd46f15) при $n = 7$: \$1,87 (CPU для GAP, 5 ядер / 28 ГиБ, 15 мин — \$0,12, `math43-c4rest-gap`; два вызова A10G, 28 + 43 мин — \$1,75, `math43-c4rest`; оба остановлены). Всего на §6.8: \$6,69.
- Ссылки († — не сверено с источником): [MP09] Mikhailov–Passi, LNM 1952 — $D_3 = \gamma_3$ †; S. A. Jennings, Canad. J. Math. 7 (1955) — рациональные размерные подгруппы †; K. W. Gruenberg, Proc. LMS (3) 7 (1957) — финитная аппроксимируемость p-группами †.

6.9. D5-7: кольцо L'_2 на четырёх образующих, проверка на компьютере; его групповой аналог

Кольцо (§6.7, последний абзац). L'_N: свободное метабелево кольцо Ли на $x_1..x_4$, порядок Холла $x_3 < x_1 < x_2 < x_4$, $t = [x_4, x_3, \dots, x_3]$ (N букв x_3), $c_i = [t, x_i]$, $u_i = [c_i, x_i]$, соотношения $r_1 = 4x_1 + 2c_3 + c_2$, $r_2 = 16x_2 + 4c_3 - c_1$, $r_3 = 64x_3 - 4c_2 - 2c_1$, (ii) $\gamma_{N+4} = 0$, (iii) все элементы базиса Холла веса $N + 3$, кроме u_1, u_2, u_3, равны 0, (iv) $u_1 = 16u_3$, $u_2 = 4u_3$, $64u_3 = 0$; $a = 32[x_1, x_2] + 64[x_1, x_3] + 128[x_2, x_3]$. Утверждалось: $D_{N+4} = \cdots = D_{2N+4} = \langle a\rangle \cong \mathbb{Z}/2$, $D_{2N+5} = 0$.

Метод (`py/lie_L2.py`, `c/lie_howell.c`). $U(L) = \mathbb{Z}\langle X\rangle$/(соотношения), поэтому $U(L)/\varpi^n = A_n/J_n$, где $J_n = \mathrm{span}\{u\rho v\}$ с обрезкой по длине n, и элемент Ли ℓ лежит в $D_n(L)$ тогда и только тогда, когда $\ell \in J_n$ (PBW: $L \to U(L)$ инъективно). $L = W/\bar{R}$, где W — линейная оболочка левонормированных скобок веса $\le N + 3$ в A_n, $\bar{R}$ — оболочка скобок $[\rho, x_{j_1}, \dots, x_{j_k}]$, обрезанных по весу $N + 3$; значит, $D_n(L) = (W \cap J_n)/\bar{R}$. *Градуировка:* при $\deg x_1 = \deg x_2 = \deg x_3 = 1$, $\deg x_4 = -N$ все соотношения однородны ($\deg c_i = \deg x_i$), и всё распадается на компоненты; a имеет степень 2. При $n \le 6$ все компоненты считаются точно над $\mathbb{Z}$ (HNF в python-flint), при $n = 7, 8$ — компонента a; при $n = 9$ компонента a (20 826 столбцов, 172 674 разреженных строк $u\rho v$) — формой Хауэлла по модулю 2^E, построенной вставкой строк на C (2 мин; свойство Хауэлла проверяется редукцией $2^{E-v}P$ для каждой ведущей строки). Ответ «$\notin$» по модулю 2^E точен; «$\in$» сертифицируется совпадением индекса при нескольких E без свободных столбцов. Метабелевы соотношения — $[U, V]$ для всех левонормированных U, V весов ≥ 2 с $w(U) + w(V) \le N + 3$ плюс все левонормированные скобки веса $N + 4$. *(Одни скобки $[[x_i, x_j], [x_k, x_l]]$ не порождают F'' как идеал Ли: с ними $x_2[c_2, t] \notin J_9$, то есть $4c_1 \ne 0$, и (3.9) при $n = 9$ не выполнялось. Это была ошибка первой версии скрипта; групповая библиотека `gap/plateau5_lib.g` делает это правильно.)*

Сверка с [IMP26, теорема A], $N = 1$ (`--var orig`, пять образующих, $\mathbb{Z}^2$-градуировка): $D_5(L_1) = \langle a\rangle \cong \mathbb{Z}/2$ (точно, все 25 компонент), $a \in D_6$ (точно), $a \notin D_7$ (по модулю 2^{16} и 2^{32}), $a \equiv c_1c_3 - c_2^2 \bmod \varpi^7$.

L'_2 [компьютер]:

n	как	результат
6	точно, все 15 компонент	$D_6 = \langle a\rangle \cong \mathbb{Z}/2$ ($a \ne 0$, $2a =$
7	точно, компонента a	$D_7 = \langle a\rangle$
8	точно (компонента a) и по модулю $2^8, 2^{16}, 2^{32}$ (индекс 745 во всех трёх, свободных столбцов нет)	$a \in D_8$, значит $D_8 = \langle a\rangle$
9	по модулю $2^8, 2^{12}, 2^{20}, 2^{32}$ (индекс 14 050 во всех, свободных столбцов нет)	$a \notin \varpi^9$, значит $D_9 = 0$

($D_7, D_8, D_9 \subseteq D_6 = \langle a\rangle$, так что начиная с $n = 7$ достаточно компоненты a.) Ещё при $n = 9$: $4c_1 = 16c_2 = 64c_3 = 0$ (3.8) и $a \equiv c_1c_3 - c_2^2$ (3.9), а $c_1c_3 - c_2^2 \notin \varpi^9$. Итак, утверждение §6.7 верно: $D_6(L'_2) = D_7 = D_8 = \langle a\rangle \cong \mathbb{Z}/2$, $D_9 = 0$, **на четырёх образующих, класс 5.** $D_9 = 0$ даёт и теорема C; вычисление с ней согласно.

Плато задаётся градуировкой [на бумаге]. В компоненте a (степень 2) у слова с k буквами x_4 остальных букв $2 + Nk$, так что его длина $2 + (N + 1)k \in \{2, N + 3, 2N + 4, \dots\}$; то же для L_N из [IMP26] с градуировкой $\deg x_5 = (0, 1)$, $\deg x_4 = (0, -N)$, $\deg x_{1,2,3} = (1, 0)$. Так как ϖ^n натянут на слова длины $\ge n$, $\varpi^{N+4} \cap U(L)_2 = \varpi^{2N+4} \cap U(L)_2$, откуда $D_{N+4} \cap L_2 = D_{2N+4} \cap L_2$: если $D_{N+4} \subseteq L_2$, длина плато $N + 1$ — это промежуток между допустимыми длинами $N + 3$ и $2N + 4$. (Посчитано: при $N = 2$ компонента a в A_6, A_7, A_8 одна и та же — 414 столбцов и 1 791 строка.) У группового кольца такой градуировки нет: $\mu(x^E) - 1 = \sum_k \binom{E}{k} a^k$ смешивает все длины.

Групповой аналог [компьютер]. Буквальный перевод `L2g` (`gap/plateau5_cands.g`, семейство `I5cand`, $d = 4$): $t = [x_4, x_3, x_3]$, $d_l = [t, x_l]$,

$$x_1^4 d_2 d_3^2, \quad x_2^{16} d_1^{-1} d_3^4, \quad x_3^{64} d_1^{-2} d_2^{-4}, \quad [d_1, x_1][d_3, x_3]^{-16}, \quad [d_2, x_2][d_3, x_3]^{-4}, \quad [d_3, x_3]^{64}, \quad [d_l, x_m]\ (l \ne m),\ [d_4, x_4],$$

$w = [x_1, x_2^{32}][x_1, x_3^{64}][x_2, x_3^{128}]$ (образ a). Тогда $G_{\mathrm{ab}} = \mathbb{Z} \oplus \mathbb{Z}/4 \oplus \mathbb{Z}/16 \oplus \mathbb{Z}/64$, как у L_2', $\gamma_5(G) \cong \mathbb{Z}/64 = \langle u_3 \rangle$, $u_3 = [d_3, x_3]$, $w = u_3^{32}$; GAP 13 мин. Тесты $u_3^{2^j}$ (CPU, `howell_gpu.py`, сертифицировано): $D_5 \cap \gamma_5 = \gamma_5$, $D_6 \cap \gamma_5 = \langle w \rangle$, $D_7 \cap \gamma_5 = 1$. **Групповой аналог останавливается на D_6, на два уровня ниже кольца.** С удвоенными или учетверёнными L_{ij} $w = 1$ уже в M.

Где ломается (лемма о спаривании из §6.7, с точностью). В кольце $e_i x_i = -C_i \in \gamma_4 \subseteq \varpi^4$, и (3.9) — это $a = \sum_i x_i g_i = \sum_i (e_i x_i)\zeta_i \in \varpi^{4+4}$ при $g_i = \sum_j L_{ij} x_j = e_i \zeta_i$. В группе из $x_i^{E_i} \in \gamma_4$ следует лишь $2^j E_i (x_i - 1) \in \Delta^{2+j}$ ($j \le 2$), и это точно, когда $\bar{x}_i$ порождает циклическое прямое слагаемое $\mathbb{Z}/o$ в G_{ab} (как в `L2g`, $o = E_i$): при отображении в C_o наименьшее F с $F(x-1) \in \Delta^k(C_o)$ равно $2^{k-2}o$ [компьютер, `scripts/cyclic_pairing_check.py`, $o = 2, \dots, 256$, $k = 2..5$; на бумаге при $k \le 4$: $o(x-1) \equiv -\binom{o}{2}(x-1)^2 \not\equiv 0$ и $2o(x-1) \equiv \frac{o(o^2-1)}{6}(x-1)^3 \not\equiv 0$ в $\Delta^k/\Delta^{k+1}(C_o) \cong \mathbb{Z}/o$]. Поэтому член $(e_i x_i)\zeta_i$ превращается в $\Delta^{2+4} = \Delta^6$ — ровно там, где останавливается `L2g`; для Δ^8 нужно $g_i \in 4E_i D$ (множитель 4, $D = \gamma_4/\gamma_5$). С другой стороны, порядок $u_{lm} = [d_l, x_m]$ делит порядок o_m элемента $\bar{x}_m$ в G_{ab} ($[d_l, x_m^{o_m}] \in [\gamma_4, \gamma_2] = 1$ в классе 5), а w — комбинация u_{lm} с коэффициентами L_{ij}/E_j; множитель поднимает L_{ij} во столько же раз. В билинейной модели двух верхних слоёв для схемы коэффициентов L_2' (`scripts/lie_L2_pairing_screen.py`: $E_i = 2^{a_i}$, $a_i \le 12$, все допустимые L_{ij}) при множителе 1 есть 84 решения с $w \ne 0$ (самое дешёвое — само L_2', $w = 8u_{22} = a$), при множителях 2 и 4 — ни одного [компьютер, модель]. (3.12) получает множитель 2 в классе 4 только за счёт пятого образующего x_5 большого порядка, который несёт w (§6.7); все попытки на четырёх образующих из §6.7 (класс 4, множитель 2) не удались, а у 80 самых дешёвых масштабирований (3.12) с множителем 4 (пять образующих) $w = 1$. [Гипотеза] метабелевой группе класса 5 с $D_8 \ne 1$, если она есть, нужен механизм, отличный от подстановки $e_i x_i \mapsto x_i^{E_i}$ (например, образующий, несущий w, порядок которого не связан с порядками спариваемых, как x_5 в (3.12)); на четырёх образующих такого у нас нет.

Тест D_9. Не запускался: группы с $w \in D_8$ не нашлось. Стоимость, если она появится ($d = 4$, $n = 9$, $M = 87\,380$): плотная форма — 30,5 ГБ (uint32), пик замыкания $\approx 1{,}64\times$ это ≈ 50 ГБ, то есть A100 80 ГБ; прогон $d = 5$, $n = 8$ ($M = 97\,655$) шёл 30 мин, так что около 30 мин, $\approx 1{,}3$ доллара по 2,5 доллара/ч, плюс сборка GAP на CPU-контейнере. Расход Modal на §6.9: 0 (всё локально).

Как повторить.

```
# точный D_6, все компоненты (нужен python-flint; 20 мин)
python3 py/lie_L2.py exact --var prime --N 2 --n 6 --full-upto 6
# компонента a:  ... --n 7 8
# сверка:        ... --var orig --N 1 --n 5 --full-upto 5
# n=9
python3 py/lie_L2.py rows --var prime --N 2 --n 9 --out F.bin
cc -O3 -o c/build/lie_howell c/lie_howell.c
c/build/lie_howell F.bin F.bin.tests 12 [1]
# тождества
python3 scripts/lie_L2_identities.py prime 2 9 T && c/build/lie_howell F.bin T 12
# группа, спецификация в results/L2/scan_L2g.jsonl (JSON группы — gpu/data/plateau5/L2g.jsonl.gz)
python3 scripts/plateau5_scan.py LOG 'L2g|I5rec(...)|5,6,7'
```

Сырые выводы — `results/L2/`.

6.10. D5-7, класс 5: все $K \subseteq \gamma_5(M)$ сразу для сдвинутого плато

Тест §6.8(b) (Предложение 4), перенесённый на группу класса 5 из §6.7 (`plateau5_c5i5`: пять образующих, $t = [x_4, x_3, x_3]$, $d_i = [t, x_i]$, девять соотношений (3.12); там $G = M/\ker\varphi$, $\gamma_5(G) \cong C_{2^{14}}$ и $w \in D_7(G) \setminus D_8(G)$, сертифицировано). Здесь Γ — это копредставление, $M = \Gamma/\gamma_6(\Gamma)\Gamma''$, $A_5 = \gamma_5(M)$, $A = \mathbb{Z}[M]$, $n \le 10$. Данные — `results/c5/kfree/`.

Лемма 5 [на бумаге] (Лемма 3 для $c = 5$). (i) $\psi : A_5 \to \Delta^5/\Delta^n$, $k \mapsto k - 1$, — гомоморфизм при $n \le 10$: $kk' - 1 = (k-1) + (k'-1) + (k-1)(k'-1)$ и $(k-1)(k'-1) \in \Delta^{10}$. (ii) Для $K \subseteq A_5$: $(K-1)A + \Delta^n = \Delta^n + \sum_k \psi(k)R$, $R = \mathrm{span}\{1\} \cup \{(x_{j_1} - 1)\cdots(x_{j_l} - 1) : 1 \le l \le n-6\}$, k по образующим K (K централен, $\psi(k)\Delta^{n-5} \subseteq \Delta^n$, а Δ/Δ^{n-5} натянуто на эти мономы). Доказательство — как у Леммы 3 с заменой $4 \to 5$.

Предложение 6 [на бумаге] (Предложение 4 для $c = 5$). Пусть $U_n = J_n + \Lambda(A_5 \otimes R_+)$, R_+ — мономы степени $1..n-6$, $\bar{\psi}_n : A_5 \to A/U_n$. Если $\bar{\psi}_n$ инъективно, то $D_n(M/K) \cap \gamma_5(M)/K = 1$ для **всех** $K \subseteq A_5$. Обратно, всякий $v \in D_n(M/K) \cap A_5$, $v \notin K$, даёт $1 \ne vk^{-1} \in \ker\bar{\psi}_n$ для некоторого $k \in K$, и $D_n(M) \cap A_5 \subseteq \ker\bar{\psi}_n$. Далее: $\ker\bar{\psi}_n$ периодично ($z \in \ker\bar{\psi}_n$ даёт $z - 1 \in \Delta^6$, т.е. $z \in D_6(M) \subseteq \sqrt{\gamma_6(M)} = \tau(M)$, Дженнингс), поэтому достаточно проверить элементы порядка 2 из

$\tau(A_5)$; $\ker\bar\psi_{n+1}\subseteq\ker\bar\psi_n$ ($n+1\le 10$); инъективность по модулю 2^{32} точна. При $n=6$ $R_+=\varnothing$ и $U_6=J_6$: тест — просто $D_6(M)\cap\gamma_5$.

Как снять $\cap\gamma_5$ [на бумаге]. Для $K\subseteq\gamma_5(M)$: $(M/K)/\gamma_5(M/K)=M/\gamma_5(M)=\Gamma/\gamma_5(\Gamma)\Gamma''=:Q$ — группа класса 4, не зависящая ни от K, ни от φ (и $G/\gamma_5(G)=Q$). Образ $D_n(M/K)$ в Q лежит в $D_n(Q)$. Итак,

$$D_n(Q)=1 \text{ и } \bar\psi_n \text{ инъективно } \implies D_n(M/K)=1 \text{ для всех } K\subseteq\gamma_5(M).$$

$D_8(Q)=1$ известно **не было**: §6.8(a) — о факторах класса 3 баз класса 4, а §6.7 проверял только $D_n(G)\cap\gamma_5$. Так как $D_8(Q)\subseteq D_5(Q)$, достаточно $D_5(Q)=1$, а по Следствию 2 (с $Q_3=Q/\gamma_4(Q)=\Gamma/\gamma_4(\Gamma)\Gamma''$) оно следует из $D_5(Q_3)=1$ (Лемма 1) и $D_5(Q)\cap\gamma_4(Q)=1$ (отображение $\gamma_4(Q)\to\Delta^4/\Delta^5$ аддитивно, ядро периодично; элементы порядка 2 должны остаться независимыми по модулю $J_5(Q)$). Отдельный тест, но дешёвый ($n=5$, $\dim A_5=780$).

Вычисление [компьютер]. `scripts/c4_q3.py gapin` теперь читает и записи `gap/plateau5_lib.g` (отбрасывает 1 250 соотношений $\ker\varphi$ и 6 250 коммутаторов веса 6, оставляя 9 соотношений Γ и 545 метабелевых); `gap/c4_q3.g` с `Q3_C` $=3,4,5$ строит Q_3, Q, M (nq на CPU-контейнере Modal, 2 ядра; с коммутаторами веса 6 на входе nq прогон класса 5 не кончился за 30 мин и был остановлен, без них, `Q3_NQTOP := false`, занял 33 мин, почти всё — GAP после nq); `howell_gpu.py batch --kfree` теперь берёт степени R_+ из класса записи ($1..n-c-1$).

группа	γ_c (ранги без кручения h)	элементов порядка 2	n	индекс J_n ($2^{32}/2^{30}/2^{20}$), сертификат
Q_3, $c=3$	$C_8^{20}C_{64}^{12}C_{1024}^{6}C_{16384}^{2}$, $h=(1,0,0)$	40	4; 5	983/977/947; 3 308/3 300/3 260, $r_n=3$,
Q, $c=4$	$C_8^{60}C_{64}^{30}C_{1024}^{12}C_{16384}^{3}$, $h=(1,0,0,0)$	105	5; 6	3 830/3 822/3 782; 13 704/13 694/13 64
M, $c=5$	$C_4^{2}C_8^{137}C_{16}C_{32}C_{64}^{57}C_{256}C_{512}C_{1024}^{18}C_{16384}^{4}$, $h=(1,0,0,0,0)$	222	6	14 722/14 712/14 662, $r_6=5$, выполне
			7	54 097/54 085/54 025, $r_7=6$, выполне

(Для M при $n=7$: U_7 добавляет 1 110 строк $\psi(b_t)a_j$ для 222 образующих b_t группы $\gamma_5(M)$, индекс $U_7=49\,496$.) Прогоны: Q_3, Q и M при $n=6$ — локально на CPU (6 с, 4,5 мин, 4 мин); M при $n=7$ — Modal A10G (194 с, пик GPU 5,2 ГиБ, ref `0ac93a8` — промежуточный коммит этой ветки с тем же кодом); прогон $n=7$ на CPU дал те же ранги и индексы в раундах 0–3 и был остановлен (`m5_n7_cpu_partial.log`). Строки: `results/c5/kfree/{q3,q4}.out.jsonl`, `m5_n6.out.jsonl`, `m5_n7_a10g.out.jsonl`.

Что это даёт.

- [компьютер + на бумаге] $D_4(Q_3)=1$ и $D_5(Q)=1$, значит $D_8(Q)=1$. Вместе с §6.7 ($D_8(G)\cap\gamma_5(G)=1$) это замыкает утверждение §6.7, что у группы класса 5 G $D_8(G)=1$: $D_8(G)$ отображается в $D_8(Q)=1$, т.е. $D_8(G)\subseteq\gamma_5(G)$. Также $D_7(G)\subseteq\gamma_5(G)$ и $D_n(M/K)\subseteq\gamma_5(M)/K$ для всех K и $n\ge 5$.
- [компьютер] При $n=6$ и $n=7$ в ядре $\bar\psi_n$ ровно один независимый элемент порядка 2, и $w\in J_n(M)$ (вместе с w_{12},w_{13},w_{23}; $u_3\notin J_n$): $w\in D_7(M)$ уже для самой M, а не только для G. Значит, $\ker\bar\psi_7$ — нетривиальная циклическая 2-группа, содержащая w, а $D_7(M)\cap\gamma_5(M)$ — её нетривиальная подгруппа. Порядок w в M не вычислялся (в G он равен 2); элемент порядка 2 ядра — $z=w^{|w|/2}$.
- [на бумаге] Поэтому весь вопрос «есть ли $K\subseteq\gamma_5(M)$ с $D_8(M/K)\ne 1$» сводится к одному биту: $\ker\bar\psi_8\subseteq\ker\bar\psi_7$ циклично, так что $\bar\psi_8$ инъективно iff $z\notin U_8$, т.е. iff `socle_bits` $=222$ при $n=8$. Если да — $D_8(M/K)=1$ для всех $K\subseteq\gamma_5(M)$: ни одна центральная модификация сдвинутого плато не даёт $D_8\ne 1$. Если нет — $z\in D_8(M/K)$ ещё не следует (тест — фильтр, как в §6.8), и следующий шаг — представление $\psi(z)=\sum_j\psi(k_j)m_j$ по модулю J_8 с $z\notin\langle k_j\rangle$ (расширенная форма Хауэлла из §6.8, не реализована).

$n=8$ [компьютер] (Modal A100-SXM4-80GB, ref `ab3023d`, `gpu/modal_batch.py spawn --gpu a100 ... -- --n 8 --kfree --verbose --group 1`, приложение `math43-c5kfree-n8`; всего 2 622 с, из них замыкание — 7 раундов, 1 881 с; пик GPU 61,0 ГиБ). $J_8(M)$: 6 804 строк соотношений, ранг 97 648, свободных столбцов $7=r_8$ при $h=(1,0,0,0,0)$, индекс 195 779 по модулю 2^{32}, 195 765 по модулю 2^{30}, 195 695 по модулю 2^{20}: разности $14=2r_8$ и $84=12r_8$, сертификат выполнен. $w,u_3,w_{12},w_{13},w_{23}\notin J_8(M)$. U_8 добавляет 6 660 строк $\psi(b_t)a_j$, $\psi(b_t)a_ja_{j'}$, индекс 170 084; 222 элемента порядка 2 группы $\gamma_5(M)$ порождают по модулю U_8 подгруппу порядка 2^{222}: $\bar\psi_8$ **инъективно** (`socle_bits` $=222$; это точно и без сертификата, который всё равно выполнен). Строка: `results/c5/kfree/m5_n8_a100.out.jsonl`.

Вывод [компьютер + на бумаге, Предложение 6, $D_8(Q)=1$ выше]. Для всех $K\subseteq\gamma_5(M)$, $M=\Gamma/\gamma_6(\Gamma)\Gamma''$, Γ — сдвинутое плато (`plateau5_c5i5`): $D_8(M/K)=1$. Значит, ни одна метабелева группа класса ≤ 5 вида Γ/N с $\gamma_6(\Gamma)\Gamma''\subseteq N\subseteq\gamma_5(\Gamma)\gamma_6(\Gamma)\Gamma''$ не имеет $D_8\ne 1$ — все центральные модификации группы §6.7 сразу, при любом φ; в частности $D_8(M)=1$, хотя $w\in D_7(M)$. Кандидата z нет: $\ker\bar\psi_8=1$. Для $D_8\ne 1$ в классе 5 это Γ исчерпано; нужно другое копредставление (ср. §6.9).

[компьютер, сертифицировано] (D5-21, 01.11.2026; принадлежности есть и в трассах, §3.2(f)). Числа битов цоколя, индексы по модулю 2^{32} и принадлежности этого раздела подтверждает `cert/check_in.c`, не делящий кода с `howell_gpu.py` (`results/cert/q{3,4,5}_n*_E32.claim`, `cert/README.md`). Он строит J_n своим разреженным замыканием; для M добавляет строки $\psi(b_t)m$ подпространства U_n; затем добавляет элементы порядка 2, проверяет свойство Хауэлла после каждого шага и то, что $2\psi(s)$ лежит в U_n (соотв. J_n) $+ 2^{32}A_n^+$ для каждого из них, и берёт число битов как разность индексов. Q_3: 40 из 40 по модулю J_4, J_5; Q: 105 из 105 по модулю J_5, J_6; M: **222 из 222 по модулю** $U_8 + 2^{32}A_8^+$, значит, $\bar\psi_8$ инъективно (полный счёт точен без сертификата), и 221 из 222 при $n = 6, 7$ (по модулю $U_n + 2^{32}A_n^+$); $w, w_{12}, w_{13}, w_{23} \in J_6, J_7$ (с проверкой точности §6.4 при $E = 32$), $\notin J_8$; индексы J_n и U_n — как выше. M при $n = 8$: 100 с и 0,7 ГБ на одном ядре kimmy (прогон на A100: 2 622 с). Не перепроверены: индексы по модулю 2^{30}, 2^{20}. На доверии: записи `gap/c4_q3.g` — все соотношения; `kgen_coeffs` — базис γ_c с порядками `ord` (проверяльщик сверяет, что строки цоколя равны (ord/2), умноженному на эти строки, так что это базис элементов порядка 2); для принадлежностей — то, что порядки $w, \ldots$ — степени двойки (они лежат в конечной $\gamma_5(M)$); лемма 5; и на бумаге предложение 6, лемма 1, следствие 2 и теорема Дженнингса. lpres подтверждает, что 40 тестов цоколя Q_3 — базис; для Q и M он не уложился в 30 мин.

Расход Modal на §6.10: \$2,56: \$0,40 на $n \le 7$ (GAP на CPU-контейнере: остановленная первая попытка \$0,12 и прогон \$0,19, `math43-c5kfree-gap`; один прогон A10G при $n = 7$ — \$0,08, `math43-c5kfree-n7`) и \$2,16 на $n = 8$ (A100-80GB, 44,5 мин с контейнером `modal_batch.py`, 4 CPU и 32 ГиБ, около 2,9 доллара/ч; `math43-c5kfree-n8`). Всё остановлено.

6.11. D5-9: на чём держится (3.12) и что нужно копредставлению класса 5

Вопрос: найти метабелеву группу класса 5 с $D_8 \ne 1$ или объяснить, почему подход §6.7–6.10 её не даёт. Здесь механизм (3.12) разложен на три части, которые проверяются по отдельности, точным тождеством в групповом кольце (Лемма 7, проверена на компьютере); каждая часть измерена в имеющихся группах (c4a = (3.12) с $x_6 \to x_3$, она же без x_5, буквальная групповая версия `L2g` кольца L'_2, сдвинутое плато `c5i5`). Данные: `results/c5/presentation/`.

Итог.

- [на бумаге] $w - 1 \equiv S - XV - \sum_{i<j} B_{ij} \pmod{\Delta^{2k}}$ (Лемма 7): S — «кольцевая часть» (спаривание), XV — коммутаторная поправка, B_{ij} — биномиальные члены степеней $x_j^{A_{ij}}$. В кольце Ли есть только S.
- [компьютер] В (3.12) пятая образующая x_5 **не** нужна для спаривания: точное спаривание $g_i \in (\gamma_3)^{2E_i}$ выполнено и с x_5, и без неё, и $S \in \Delta^6$ в обоих случаях. Она нужна для биномиальной ошибки: без x_5 (или без её соотношения $x_5^{1024}d_2^{16}$) $B_{12} \notin \Delta^6$, точнее $128(x_2-1)^2(x_1-1) \notin \Delta^6$; с ней $B_{12} \in \Delta^6$ и $B_{13} + B_{23} \in \Delta^6$ (по отдельности — нет).
- [на бумаге] Два препятствия в классе 5, оба величиной $c-3$ уровня: (i) спаривание требует множителя $2^{k-2} = 4$ (§6.7, §6.9); (ii) для пары, несущей w ($A_{ij} < E_iE_j$ вынуждено, Лемма 9), лемма о спаривании гарантирует биномиальную ошибку лишь до Δ^{c+1} (Лемма 10), а для Δ^{2c-2} нужно ещё $c-3$ уровня от других соотношений. Наблюдение [компьютер]: без носителя w доходит ровно до D_{c+1} (c4b: D_5; `L2g`: D_6), с одним носителем — до D_{c+2} (c4a: D_6; `c5i5`: D_7).
- [компьютер] Не работают три новых семейства: «расщепление $(c-2, c)$» ($w = 1$ во всех 62 построенных группах), корневые расширения $x_i \mapsto y_i^m$ ($w = 1$) и решения билинейной модели с равными порядками (в 3 из 4 $w = 1$, в четвёртой все $g_i = 1$ и $w \in D_5 \setminus D_6$).
- [гипотеза] Для $D_8 \ne 1$ в классе 5 нужны два независимых механизма-носителя, т.е. не меньше шести образующих. Плотная проверка такой группы при $n = 8$ недоступна ($\dim A_8 = 335\,923$ при $d = 6$).

(a) Разложение Обозначения. G — любая группа, Δ — её идеал аугментации, $X_i = x_i - 1$ для $x_1, x_2, x_3 \in G$; A — кососимметричная целочисленная матрица с $h_{ij} := x_j^{A_{ij}} \in \gamma_k(G)$ $(i \ne j)$; $g_i = \prod_{j\ne i} h_{ij}$, $u_{ij} = [x_i, h_{ij}]$ ($[a,b] = a^{-1}b^{-1}ab$), $v_i = \prod_{j>i} u_{ij}$, $w = \prod_{i<j} u_{ij} = [x_1, x_2^{A_{12}}][x_1, x_3^{A_{13}}][x_2, x_3^{A_{23}}]$ (тот самый w из (3.12), §6.7, `L2g`), и

$$S = \sum_i X_i(g_i - 1), \qquad XV = \sum_i X_i(v_i - 1), \qquad B_{ij} = \sum_{m\ge 2} \Big[\tbinom{A_{ij}}{m} X_j^m X_i + \tbinom{-A_{ij}}{m} X_j X_i^m\Big].$$

Лемма 7 [на бумаге]. $w - 1 \equiv S - XV - \sum_{i<j} B_{ij} \pmod{\Delta^{2k}}$. *Доказательство.* $(h-1)(h'-1) \in \Delta^{2k}$ для $h, h' \in \gamma_k$, поэтому $g_i - 1 \equiv \sum_j H_{ij}$, $H_{ij} = h_{ij} - 1$. Из $x_i h = h x_i [x_i, h]$: $X_iH_{ij} - H_{ij}X_i = hx_i(u_{ij}-1) = (u_{ij}-1) + (X_i + H_{ij} + H_{ij}X_i)(u_{ij}-1)$, и $u_{ij} \in \gamma_{k+1}$, так что $X_iH_{ij} \equiv H_{ij}X_i + (u_{ij}-1) + X_i(u_{ij}-1)$. Далее $H_{ij}X_i + X_jH_{ji} = \sum_{m\ge1} \binom{A}{m} X_j^m X_i + \sum_{m\ge1} \binom{-A}{m} X_j X_i^m$ ($A = A_{ij}$, $H_{ji} = x_i^{-A} - 1$; для отрицательного показателя биномиальный ряд конечен по модулю Δ^{2k}), и члены $m = 1$ сокращаются: это

B_{ij}. Сумма по $i<j$: $S \equiv \sum_{i<j}(u_{ij}-1) + XV + \sum B_{ij}$; наконец, $\sum_{i<j}(u_{ij}-1) \equiv w-1$, так как $(u-1)(u'-1) \in \Delta^{2k+2}$, и $\sum_{j>i} X_i(u_{ij}-1) \equiv X_i(v_i-1)$. □

От G не требуется ничего, кроме $h_{ij} \in \gamma_k$ (даже метабелевости). В кольце Ли аналог точен и $XV = B = 0$: $a = \sum_{i,j} A_{ij}x_ix_j = \sum_i x_ig_i$ [IMP26, (3.9)]. Итак, $w \in D_{2k}$ iff $S - XV - \sum B_{ij} \in \Delta^{2k}$, и три части — три места, где группа может проиграть кольцу уровни.

Лемма 8 [на бумаге] (достаточные условия). Пусть $x_i^{E_i} \in \gamma_k$, $E_i = 2^{a_i} \geq 4$, $k \leq 4$ (так что верна лемма о спаривании из §6.7: $2^sE_iX_i \in \Delta^{2+s}$ при $2+s \leq k$). (i) Если $g_i = \zeta^F s^{F'}$, $\zeta \in \gamma_k$, $s \in \gamma_{k+1}$, $F = 2^{k-2}E_i$, $F' = 2^{k-3}E_i$, то $X_i(g_i-1) \in \Delta^{2k}$. (ii) Если $v_i \in (\gamma_{k+1})^{2^{k-3}E_i}$, то $X_i(v_i-1) \in \Delta^{2k}$. *Доказательство.* $\zeta^F - 1 \equiv F(\zeta-1)$ по модулю Δ^{2k}, и $X_i \cdot F(\zeta-1) = (FX_i)(\zeta-1) \in \Delta^{k+k}$; так же $X_i(s^{F'}-1) \equiv (F'X_i)(s-1) \in \Delta^{(k-1)+(k+1)}$; перекрёстный член $(\zeta^F-1)(s^{F'}-1) \in \Delta^{2k+1}$. (ii) — вторая половина (i). □

Это «точное спаривание»: множитель 2^{k-2} на γ_k-части g_i (2 для (3.12), 4 в классе 5) и 2^{k-3} на её γ_{k+1}-части. Условие (ii) выполнено, как только в верхнем слое есть элементы большого порядка (в G из (3.12) и из §6.7 $\gamma_c(G) = C_{2^{14}}$).

Лемма 9 [на бумаге]. Пусть G метабелева класса c, $k = c-1$, $x_i^{E_i} = P_i \in \gamma_k$ и $E_iE_j \mid A_{ij}$. Тогда $u_{ij} = 1$. Значит, $w \neq 1$ требует $E_iE_j \nmid A_{ij}$ для некоторой пары $i<j$ (для степеней двойки: $A_{ij} < E_iE_j$). *Доказательство.* $u_{ij} = [x_i, P_j^q] = [x_i, P_j]^q$, $q = A_{ij}/E_j$, так как $[x_i, P_j] \in \gamma_c$ централен; и $[x_i, P_j]^{E_i} = [x_i^{E_i}, P_j] = [P_i, P_j] = 1$ ($\gamma_k \subseteq G'$ абелева). □

(b) Какая часть ломается: измерение `scripts/c5_lintests.py` записывает S, Err $= S-(w-1)$, XV, B_{12}, B_{13}, B_{23}, $\sum B$, $XV + \sum B$ и chk $=$ Err $- XV - \sum B$ как целочисленные комбинации $\sum_k c_k(w_k-1)$ слов (поле `lintests` записи); `gpu/howell_gpu.py` (новое: строки $\sum_k c_k(\mu(w_k)-1)$ таких комбинаций, выводятся после обычных тестов) даёт их принадлежность J_n, т.е. Δ^n; `scripts/c5_lintests_extra.py` добавляет любой многочлен от X_i. Все прогоны локальные (CPU), все сертификаты выполнены (свободных столбцов $= r_n$, разности индексов $2r_n$, $12r_n$).

группа	c, k	n	w	S	XV	B_{12}	B_{13}	B_{23}	$\sum B$	chk
c4a: (3.12), $x_6 \to x_3$ (`plateau5_c4i5`)	4, 3	5	∈	∈	∈	∈	∈	∈	∈	∈
		6	∈	∈	∈	∈	∉	∉	∈	∈
		7	∉	∉	∈	∉	∉	∉	∉	∈
c4b: c4a без x_5	4, 3	5	∈	∈	∈	∈	∈	∈	∈	∈
		6	∉	∈	∈	∉	∉	∉	∉	∈
c4a без $x_5^{1024}d_2^{16}$	4, 3	6	∉	∈	∈	∉	∉	∉	∉	∈
`L2g` (§6.9)	5, 4	6	∈	∈	∈	∈	∈	∈	∈	∈
		7	∉	∉	∈	∉	∉	∉	∉	∈
`c5i5` (§6.7)	5, 4	8	∉	∉	∈	∉	∉	∉	∉	∈

chk $\in \Delta^n$ везде, как и утверждает Лемма 7. Куски $B_{12} \equiv 128(X_2X_1^2 - X_2^2X_1)$ (по модулю Δ^6, Лемма 10 ниже):

группа	n	$128X_2X_1^2$	$128X_2^2X_1$
c4a	6	∈	∈
c4a	7	∈	∉
c4b	6	∈	∉
c4a без $x_5^{1024}d_2^{16}$	6	∈	∉
`c5i5`	8	∈	∉

(Контроль: $16X_1$, $128X_2$, $8X_1^2$, $64X_2^2 \notin \Delta^6$ в обеих, как и должно быть.)

Точное спаривание (`gap/c5_ep.g`, в $M = \Gamma/\gamma_{c+1}\Gamma''$ и в $G = M/\ker\varphi$; наибольшее a с $g_i \in (\gamma_k)^{2^a}\gamma_c$ и правильна ли γ_c-часть, Лемма 8(i)) [компьютер]:

группа	g_1	g_2	g_3
c4a ($E = 8, 64, 16384$)	ровно $(\gamma_3)^{16}$: множитель 2	ровно $(\gamma_3)^{128}$: 2	$g_3 = 1$
c4b	так же	так же	$g_3 = 1$

группа	g_1	g_2	g_3
`c5i5` (класс 5, $k=4$)	$(\gamma_4)^{16}$, но не $(\gamma_4)^{32}\gamma_5$: множитель 2, нужно 4	только $(\gamma_4)^{128}$: 2, нужно 4	$g_3=1$

(То же в M и в G для всех трёх; w имеет порядок 2 в M во всех трёх. Журналы `ep_c4a.log`, `ep_c4b.log`, `ep_c5i5.log`.)

Итак, в (3.12) спаривание x_5 вообще не видит (те же множители, та же точность, в M и в G), и $S\in\Delta^6$ с ней и без неё. **Что делает** x_5 [компьютер]: переводит $128X_2^2X_1 = 2E_2X_2^2X_1$ в Δ^6 (одна лемма о спаривании даёт Δ^5), отсюда $B_{12}\in\Delta^6$, и делает $B_{13}+B_{23}\in\Delta^6$, хотя ни одно слагаемое там не лежит. Если убрать только её степенное соотношение $x_5^{1024}d_2^{16}$, первый эффект пропадает. Подъём ровно на один уровень: при $n=7$ в c4a $128X_2^2X_1\notin\Delta^7$ (CPU, 3,8 ч на загруженной машине, сертификат выполнен; `c4a_lin2_n7.log`). (Какое тождество в $\mathbb{Z}[G]$ это делает, на бумаге не найдено.) `L2g` ломается при $n=7$ на S (множитель 1 там, где нужен 4) и на каждом B_{ij}.

(c) Подсчёт уровней: почему носитель покупает один уровень *Гарантированный уровень* монома $NX_{t_1}\cdots X_{t_m}$ (N — степень двойки) — наибольшая сумма $\sum_r \ell_r$ по разложениям $N=\prod_r N_r$, где ℓ_r — уровень, который лемма о спаривании даёт $N_rX_{t_r}$ (1, если $N_r<E_{t_r}$, иначе $\min(2+\log_2(N_r/E_{t_r}),k)$). Это нижняя оценка истинного уровня; соотношения могут дать больше.

Лемма 10 [на бумаге]. Пусть $E_i\le E_j$, $A=A_{ij}=2^\alpha<E_iE_j$ (пара из Леммы 9). Гарантированный уровень $\binom{A}{2}X_j^2X_i$ не больше $k+2=c+1$. *Доказательство.* $v_2\binom{A}{2}=\alpha-1\le a_i+a_j-2$. Поднять X_j до уровня ≥ 2 стоит $\ge a_j$, и остаётся $\le a_i-2<a_i$: два других множителя остаются на уровне 1, всего $\le k+2$. Иначе поднимается только X_i, до $\le k$: всего $\le k+2$. □

Тот же подсчёт даёт S гарантированный уровень $k+\min(2+\log_2 m,k)$ при множителе m, а XV — уровень $2k$ при Лемме 8(ii). Значит, одной леммой о спаривании разложение гарантирует в лучшем случае $w\in D_{c+1}$, а для D_{2c-2} нужно ещё $c-3$ уровня для биномиального члена w-пары: один в классе 4, два в классе 5. Данные ложатся точно:

группа	c	носитель	множитель	гарантировано $(S;B_w)$	наблюдается
c4b, c4a без соотношения x_5	4	нет	2	6; 5	$w\in D_5\setminus D_6$
c4a ((3.12))	4	x_5	2	6; 5 (+1 от x_5, измерено)	$w\in D_6$, $D_7(M/K)=1$ (§6.8)
`L2g`	5	нет	1	6; 6	$w\in D_6\setminus D_7$
`c5i5` (§6.7)	5	x_5	2	7; 6	$w\in D_7\setminus D_8$

[гипотеза H] Соотношение-носитель вроде $x_5^{1024}d_2^{16}$ поднимает биномиальную ошибку w-пары на один уровень, не больше; тогда для D_8 в классе 5 нужны множитель 4 *и* два таких механизма. В пяти образующих (больше плотный тест при $n=8$ не вмещает) место есть для одного.

(d) Что нужно копредставлению класса 5 При $k=c-1=4$, по Леммам 7–10 [на бумаге]:

- (R1) спаривание: $g_i\in(\gamma_4)^{4E_i}(\gamma_5)^{2E_i}$ для всех i — или сокращение $S\equiv XV+\sum B$ по модулю Δ^8 иного рода;
- (R2) $v_i\in(\gamma_5)^{2E_i}$: верхний слой с элементами порядка $\gg E_i$ (циклическая $\gamma_5(G)$ большого порядка, как в §6.7);
- (R3) биномиальная ошибка w-пары в Δ^8: на два уровня выше гарантированного леммой о спаривании;
- (R4) $w\ne 1$: $A_{ij}<E_iE_j$ для некоторой пары (Лемма 9) — именно это делает трудным (R3) (Лемма 10), а в шаблоне L_2' — ещё и невозможным (R1) (§6.9: при множителях 2 и 4 решений нет).

Билинейная модель §6.9 со *свободными* коэффициентами p_{il} решения с множителем 4 имеет. [на бумаге] Если в соотношениях только d_2 ($x_1^{E_1}=d_2^{p_{12}}$, $x_3^{E_3}=d_2^{p_{32}}$), условия сводятся к $v_2(q_{21}p_{12})=v_2(q_{23}p_{32})<a_2$ и двум неравенствам, верным при большом A_{13}; при $E_1=E_2=E_3$ множитель вообще не входит (d_l имеют порядок $\le E_l$, и $\gcd(4E_i,E_l)=E_l$). [компьютер, `scripts/c5_split_screen.py`] При равных порядках $E\le 2^6$, $p_{il}\in\{0,\pm1,\pm2\}$: 40 решений, самое дешёвое — $E=4$, $A=(4,8,4)$, $p_{12}=\pm2$, $p_{32}=\pm2$. Но «$w\ne 0$» модели ненадёжно: из четырёх таких групп, построенных в GAP (`eq4a`, `eq4b`, `eq8a`, `eq8b`: $E=4$ или 8 у всех трёх, $P_1=d_2^{\pm2}$, $P_3=d_2^2$, класс 5, $t=[x_4,x_3,x_3]$), в трёх $w=1$ в M, а в `eq8a` (w порядка 2) все $g_i=1$ точно — $S=0$, w — чистая биномиальная ошибка, и $w\in D_5\setminus D_6$ (сертифицировано) [компьютер].

(e) Другие кандидаты и почему они не работают

1. **Расщепление** $(c-2, c)$**.** Спаривать на весе $c-2$, а второй множитель брать в центре: если $x_i^{E_i} \in \gamma_{c-2}$ и $g_i \in (\gamma_c)^{2^{c-4}E_i}$, то $X_i(g_i-1) \in \Delta^{2c-2}$ (как в Лемме 8: $2^{c-4}E_iX_i \in \Delta^{c-2}$), с множителем 2 в классе 5, как в (3.12). (Лемма 7 тогда верна лишь по модулю Δ^{2c-4}; члены ошибки больше.) [на бумаге, набросок] В кольце Ли это не может нести D_{2c-2}: ретракция из доказательства [IMP26, теорема C] (на T, нулевая на $Z(T)\cdot T$) убивает $\sum_i (e_i x_i)\zeta_i$ с центральными ζ_i и оставляет a на месте; в сдвинутом семействе γ_c-часть g_i берётся только из биномов ($E_l d_l = -\binom{E_l}{2}[d_l, x_l]$), так что в кольце она нулевая. [компьютер] Лабораторная проверка в классе 4 (расщепление $(2,4)$): $d = [x_4, x_a]$ или $[x_a, x_b]$, v веса 4 от x_3, x_4, $x_i^{E_i} = d^{p_i}$, $d^o = v^s$; 60 случайных наборов, проходящих модель с одним d, `scripts/c5_split_gen.py 4 60 1` = `gap/c5_spl_c4_1.g`, `gap/c5_split.g`): $w = 1$ в M во всех 60. Явные попытки `lab24a`, `lab24b`: $w = 1$.
2. **Корневые расширения** $x_i \mapsto y_i^m$ (`gap/c5_roots.g`). Для $x = y^m$ множитель падает [компьютер, `scripts/c5_cyclic_power_check.py`: в $\mathbb{Z}[C_N]$ для $x = g^m$ порядка E условие $F(x-1) \in \Delta^k$ требует $F/E = 2^{k-2}$ при $m = 1$, $(1,1,2,2)$ при $m = 2$ и 1 при $m \geq 4$, $k = 2..5$]. Но это замена координат, а не новый механизм: в кольце a умножается на m^2, а порядки — только на m, и a умирает [на бумаге]. В GAP: у `L1` (групповая версия L_1, §6.6) w порядка 2 в M, у её расширения квадратными корнями `L1r2` $w = 1$ в M [компьютер].
3. **Два носителя** (предложение, не построено). По гипотезе H: (3.12), сдвинутое в класс 5, со вторым носителем x_6, повторяющим роль x_5 для оставшегося уровня. Шесть образующих: плотный тест при $n = 8$ имеет $\dim A_8 = 335\,923$ (около 450 ГБ в `uint32`), что не помещается ни на один GPU; даже $n = 7$ (55 986) покажет только D_7. Статус: [гипотеза], не проверено.

(f) В сторону $f_{\text{met}}(5) = 7$ Доказано утверждение о методе, а не о группах: для копредставления вида (3.12) (спаренные образующие с $x_i^{E_i} \in \gamma_{c-1}$, $w = \prod[x_i, x_j^{A_{ij}}]$) $w-1$ раскладывается по Лемме 7; $w \neq 1$ требует пары с $A_{ij} < E_iE_j$ (Лемма 9); для этой пары лемма о спаривании гарантирует биномиальную ошибку лишь до Δ^{c+1} (Лемма 10), а кольцевую часть — до $\Delta^{k+\min(2+\log_2 m, k)}$. Значит, любое доказательство $w \in D_8$ в классе 5 в этом семействе должно получить два уровня от соотношений, отличных от степенных соотношений спаренных образующих, плюс множитель 4 (или сокращение между S и ошибками). Во всех измеренных группах выигранных уровней ровно столько, сколько носителей (таблица в (c)). Для теоремы $f_{\text{met}}(5) = 7$ нужна оценка сверху на D_8, не проходящая через это разложение; здесь её нет.

(g) `c5i5` при $n = 8$**: ломается и на** S**, и на** B [компьютер] (Modal A100-SXM4-80GB, main 98e827c, `C4BATCH_APP=math144-c5i5-n8`, `gpu/modal_batch.py spawn --gpu a100 --input results/c5/presentation/records/c5i5_ -- --n 8 --h 1 0 0 0 0 --verbose`; 1 567 с, замыкание за 7 раундов, пик GPU 54,8 ГиБ.) $M = 97\,655$, 8 054 строк соотношений, ранг 97 648, свободных столбцов $7 = r_8$, индекс 169 278 / 169 264 / 169 194 по модулю $2^{32}/2^{30}/2^{20}$ (разности $14 = 2r_8$, $84 = 12r_8$): сертификат выполнен. Строки для таблиц (b): $w, S \notin \Delta^8$, $XV \in \Delta^8$, $B_{12}, B_{13}, B_{23}, \sum B \notin \Delta^8$, $\text{chk} \in \Delta^8$; $128X_2X_1^2 \in \Delta^8$, $128X_2^2X_1 \notin \Delta^8$. Строка и журнал: `results/c5/presentation/c5i5_lin_n8_a100.out.jsonl`, `.log`.

Итак, `c5i5` ломается на D_8 ровно так же, как `c4a` и `L2g` на D_7: в части спаривания S (множитель 2 вместо нужных 4, (b)) **и** в биномиальной ошибке w-пары, через тот же моном $2E_2X_2^2X_1$. Носитель x_5 не поднимает биномиальную ошибку с гарантированного $\Delta^{c+1} = \Delta^6$ до Δ^8: не больше одного уровня, как и в классе 4 (при $n = 7$ части не измерялись; там известно только $w \in D_7$, §6.10). Это предсказание гипотезы H; альтернатива «только S» (H неверна) исключена. H остаётся гипотезой (две группы, по одной на класс).

Дальше. (1) Сделано: (g). (2) Тождество, стоящее за «x_5 поднимает $128X_2^2X_1$», на бумаге — оно сказало бы, каким должен быть второй носитель (сделано: §6.12). (3) Для проверки D_8 на шести образующих нужен метод, отличный от плотной формы (стандартные мономы или градуировка).

Перезапуск.

```
# запись c4a (и запись c4b из таблицы)
scripts/gap5.sh c5_ep.g 'P5NAME := "c4a"; P5I5 := rec(cls := 4);'
python3 scripts/c5_lintests.py IN OUT --A 256 16384 32768
python3 scripts/c5_lintests_extra.py IN OUT 'T12=128*X2*X2*X1'
python3 gpu/howell_gpu.py --device numpy plateau --input OUT.gz --n 6 --h 1 0 0 0 --steps host
# расщепления и корневые расширения
mkdir -p gpu/data/plateau5/spl && scripts/gap5.sh c5_split.g 'Read("c5_spl_eq.g"); SPLDIR := "/data/spl";'
scripts/gap5.sh c5_roots.g 'ROOTBASE := "L1"; ROOTM := 2;'
```

Журналы и строки: `results/c5/presentation/`; записи с `lintests`: `results/c5/presentation/records/`.

Расход Modal на §6.11: около $1,4 на (g) (один прогон A100-80GB, 26 мин с контейнером, приложение `math144-c5i5-n8`, остановлено); остальное локально (GAP в Docker, замыкания на CPU).

6.12. D5-10: тождество носителя, почему он даёт один уровень и что нужно конструкции класса 5

Вопрос §6.11(b): какое тождество в $\mathbb{Z}[G]$ кладёт $128X_2^2X_1$ в Δ^6 в c4a ((3.12) с $x_6 \to x_3$), причём только при соотношении носителя $x_5^{1024}d_2^{16}$. Обозначения §6.11: $t = [x_4, x_3]$ (класс 4) или $[x_4, x_3, x_3]$ (класс 5), $d_l = [t, x_l] \in \gamma_k$, $k = c-1$, $X_i = x_i - 1$, $D_l = d_l - 1$, $E_i = 2^{e_i}$ — порядок x_i по модулю γ_k, $P_i = x_i^{E_i} \in \gamma_k$. В c4a: $P_1 = d_2^{-1}d_3^{-4}$, $P_2 = d_1d_3^{16}d_5^{-16}$, $P_3 = d_1^4d_2^{-16}$, $x_5^{1024} = d_2^{-16}$, $E = (8, 64, 16384)$, $(A_{12}, A_{13}, A_{23}) = (256, 16384, 32768)$. Данные: `results/c5/carrier/`.

Итог.

- [на бумаге, каждый шаг проверен на компьютере] Тождество короткое. Перенести скаляр $128 = 2E_2 = 16E_1$ с X_2 на X_1:
$$128X_2^2X_1 \equiv X_2^2\,(x_1^{128} - 1), \qquad x_1^{128} = P_1^{16} = d_2^{-16}d_3^{-64} = x_5^{1024}\,d_3^{-64} \pmod{\Delta^6},$$
далее $X_2^2(x_5^{1024} - 1) \equiv 1024X_2^2X_5 \in \Delta^6$, потому что $512X_2^2 \in \Delta^5$ (Лемма 13, «квадратичное спаривание»), и $64X_2^2D_3 = (64X_2)X_2D_3 \in \Delta^{2+1+3}$. Используются только соотношения x_1, x_2, x_5 и класс ≤ 4 (Теорема 14). Без носителя те же шаги приводят к $-16X_2^2D_2$, а это $\notin \Delta^6$ [компьютер].
- [на бумаге] Итак, носитель превращает d_2-часть d_2^{-16} элемента x_1^{128} — *малую* степень элемента γ_k, которую X_2^2 гарантирует лишь до Δ^5, — в *большую* степень x_5^{1024} образующей, которую поглощает квадратичное спаривание. d_3-часть d_3^{-64} не переносится: обычное спаривание кладёт её в Δ^{k+3}, а это Δ^6 в классе 4.
- [на бумаге] Лемма 15: если спаривание выполнено с множителем 1, то $w = [x_2, x_1^{A_{12}}] = \prod_l [x_2, d_l]^{-a_lA_{12}/E_1}$ ($P_1 = \prod_l d_l^{-a_l}$), и всякая d_l-часть, которую спаривание гарантирует само, даёт 1. Значит, для $w \ne 1$ нужна часть, которую спаривание не гарантирует, т.е. перенесённая носителем: носитель вынужден, а не случаен.
- (b) [на бумаге] Только один уровень: непереносимая часть $64X_2^2D_3$ стоит ровно на Δ^{k+3} ($= \Delta^6$ в c4a); всё остальное в тождестве гарантировано на уровень выше. Предсказание «при $n = 7$ ломается кусок $64X_2^2D_3$, а кусок носителя и биномы x_1 — нет» подтверждено [компьютер, Modal A10G, сертифицировано]: тождество верно по модулю Δ^7, и вне Δ^7 только $64X_2^2D_3$ (см. (b)).
- (c) [на бумаге] Условия для $NX_2^2X_1 \in \Delta^{2k}$ ($N = A_{12}/2$): каждая d_l-часть x_1^N либо переносится степенью x_κ^F образующей с $2^kE_2 \mid F$, либо имеет $v_2(a_lN/E_1) \ge e_2 + k - 3$ (Предложение 16, достаточное условие); второй носитель нужен, когда два куска не выполняют второе условие. В `c5i5` не выполняет d_3-часть ($64 < 2E_2$) — поэтому там $128X_2^2X_1 \notin \Delta^8$ (§6.11(g)). [компьютер] Построено семь копредставлений: второй носитель на `c5i5` (`c5i6`, шесть образующих), две конструкции с множителем 4 и четыре варианта с одним изменением — $w = 1$ **в** $M = \Gamma/\gamma_6\Gamma''$ **во всех** ((d)). Единственной группой этого семейства с $w \ne 1$ в классе 5 остаётся `c5i5`.

(a) Тождество Какие соотношения его несут [компьютер, `scripts/c5_carrier_ablate.sh`, $n = 6$, CPU]. В тестовой записи §6.11 (c4a $= G$ из `plateau5_c4i5`, 1 554 соотношения: 9 из (3.12), 45 метабелевых, 250 из $\ker\varphi$, 1 250 коммутаторов веса 5) удаление пяти соотношений $d_l^{N_l}$ и двух верхних, или 45 метабелевых, или 250 из $\ker\varphi$ сохраняет $128X_2^2X_1 \in \Delta^6$ (сертифицировано); удаление соотношения x_3 тоже сохраняет (принадлежность найдена; для сертификата нужно другое h, не переделывалось); удаление соотношения x_1 или x_2 его теряет (не сертифицировано по той же причине), как и удаление соотношения x_5 (§6.11, сертифицировано). Перекладывание скаляра: в c4a, c4b и c4a без соотношения x_5 ($h = (2, 1, 2, 0)$, $r_6 = 47$, сертификат выполнен), с $e_1 = 2D_1X_2X_1$, $e_2 = 32D_3X_2X_1$, $e_3 = 32D_5X_2X_1$, $e_4 = 32X_2^4X_1$: $128X_2^2X_1 \equiv e_1 + e_2 - e_3 + 3969\,e_4$ (соотношение x_2), $e_2, e_3, e_4 \in \Delta^6$ во всех трёх, а $e_1 \in \Delta^6$ только в c4a: весь эффект сидит в $2D_1X_2X_1$, а это снова $128X_2^2X_1$, прочитанное через соотношение x_2. Выход — читать скаляр через соотношение x_1.

Лемма 13 [на бумаге] (квадратичное спаривание). Пусть $k \in \{3, 4\}$, G класса $\le k+1$, $x^E = P = \prod_l d_l^{e_l}$, $d_l \in \gamma_k$, $E = 2^e \ge 4$, $X = x - 1$, $s \in \{0, 1\}$, $2 + s \le k$. Пусть для каждого l либо $E \mid 8e_l$, либо $d_l^{2^{s+3}e_l} \in \gamma_{k+1+s}$. Тогда $2^{s+3}EX^2 \in \Delta^{k+2+s}$.

Доказательство. $x^E - 1 = \sum_{m\ge1}\binom{E}{m}X^m$, поэтому $2^{s+3}EX^2 = 2^{s+3}(P-1)X - \sum_{m\ge2}2^{s+3}\binom{E}{m}X^{m+1}$. По модулю Δ^{2k} $P - 1 \equiv \sum_l e_lD_l$. Член $2^{s+3}e_lD_lX$ равен $D_l \cdot \frac{8e_l}{E}\,(2^sEX) \in \Delta^{k+2+s}$ в первом случае (лемма о спаривании §6.7: $2^sEX \in \Delta^{2+s}$) и $(d_l^{2^{s+3}e_l} - 1)X \in \Delta^{k+2+s}$ с точностью до Δ^{2k} во втором. При $m = 2$: $2^{s+2}E(E-1)X^3$ есть $(4EX)X^2 \in \Delta^{\min(4,k)+2} = \Delta^{k+2}$ при $s = 0$ и $(E-1)X(8EX^2) \in \Delta^{k+3}$ при $s = 1$ (по случаю $s = 0$). При $m \ge 3$ $v_2\binom{E}{m} = e - v_2(m)$, и член — обратимое кратное $(2^{s+3-v_2(m)}EX)X^m$ уровня $\ge \min(2 + \max(0, s + 3 - v_2(m)), k) + m \ge k + 2 + s$. □

Откуда второй случай [на бумаге]: если $t \in \gamma_{k-1}$, $d_1 = [t, x_1]$ и $x_1^{E_1} \in \gamma_k$, то $1 = [t, x_1^{E_1}] = d_1^{E_1}[d_1, x_1]^{\binom{E_1}{2}}$ (класс $k+1$, $[\gamma_{k-1}, \gamma_k] \subseteq \gamma_{2k-1} = 1$), т.е. $d_1^{E_1} \in \gamma_{k+1}$, а $[d_1, x_1]^{E_1} = [d_1^{E_1}, x_1] = 1$ даёт $d_1^{2E_1} = 1$. Для c4a ($x = x_2$, $E = 64$, $e_1 = 1$, $e_3 = 16$, $e_5 = -16$, $E_1 = 8$): $512X_2^2 \in \Delta^5$ и $1024X_2^2 \in \Delta^6$. Измерено [компьютер]: $2^s X_2^2 \in \Delta^5$ тогда и только тогда, когда $s \geq 9$, причём $512X_2^2 \in \Delta^6$ уже (лучше гарантированного); $2^s X_2 \notin \Delta^4$ при всех $s \leq 14$ — выигрыш действительно квадратичный, а не лучшее линейное спаривание.

Явный вид при $n = 5$ [компьютер, `scripts/c5_carrier_rows.py --ddmin`, `scripts/c5_carrier_solve.sage`]: хватает 14 сдвинутых строк соотношений (только $r_1 = x_1^8 d_2 d_3^4$ и $r_2 = x_2^{64} d_1^{-1} d_3^{-16} d_5^{16}$, ни одного коммутаторного), и над $\mathbb{Z}$

$$512X_2^2 \equiv 8X_2(r_2-1) - 252X_2^2(r_2-1) + 2730X_2(r_2-1)X_2^2 - 2(r_2-1)\big(\Lambda_3 + \Lambda_5\big) + X_2\big[(r_1-1),\, X_3X_4 - X_4X_3\big] \pmod{\Delta^5},$$

$\Lambda_3 = -X_3X_3X_4 + 2X_3X_4X_3 - X_4X_3X_3$, $\Lambda_5 = -X_3X_4X_5 + X_4X_3X_5 + X_5X_3X_4 - X_5X_4X_3$ (старшие формы D_3, D_5 с точностью до знака), $[a,b] = ab - ba$: три ингредиента доказательства — соотношение x_2 с биномиальными поправками, $2D_l \cdot 64X_2$ и $[t, x_1^8] = 1$ — видны буквально (это зеркальный вариант, с X_2 слева).

Теорема 14 [на бумаге]. Пусть G класса ≤ 4, $t \in \gamma_2$, $d_l = [t, x_l]$ и $x_1^8 = d_2^{-1} d_3^{-4}$, $x_2^{64} = d_1 d_3^{16} d_5^{-16}$, $x_5^{1024} = d_2^{-16}$. Тогда $128X_2^2X_1 \in \Delta^6$.

Доказательство. (1) $128X_2^2X_1 = X_2^2(x_1^{128} - 1) - \sum_{m=2}^{5}\binom{128}{m}X_2^2X_1^m$. Сумма лежит в Δ^6: $\binom{128}{2} = 127 \cdot 64$ и $64X_1^2 = (8X_1)^2 \in \Delta^4$; $\binom{128}{3} = 2^7 \cdot 2667$ и $128X_1^3 = (8X_1)(16X_1)X_1 \in \Delta^6$; при $m \geq 4$ степень ≥ 6. (2) γ_3 абелева ($[\gamma_3, \gamma_3] \subseteq \gamma_6$), поэтому $x_1^{128} = (d_3^{-4}d_2^{-1})^{16} = d_3^{-64}d_2^{-16} = d_3^{-64}x_5^{1024}$ и $x_1^{128} - 1 = (d_3^{-64} - 1) + (x_5^{1024} - 1) + (d_3^{-64} - 1)(x_5^{1024} - 1)$, последний член в $\Delta^3\Delta^3$. (3) $X_2^2(d_3^{-64} - 1) \equiv -64X_2^2D_3 = -(64X_2)X_2D_3 \in \Delta^{2+1+3}$. (4) $X_2^2(x_5^{1024} - 1) = \sum_{m\geq 1}\binom{1024}{m}X_2^2X_5^m$: при $m = 1$ это $2(512X_2^2)X_5 \in \Delta^6$ по Лемме 13; $\binom{1024}{2} = 512 \cdot 1023$ даёт $(512X_2^2)X_5^2 \in \Delta^7$; $\binom{1024}{3}$ делится на 1024; при $m \geq 4$ степень ≥ 6. □

То есть как тождество:

$$128X_2^2X_1 \;\equiv\; \underbrace{X_2^2(x_5^{1024}-1)}_{\text{носитель, Лемма 13}} \;-\; \underbrace{64X_2^2D_3}_{\text{спаривание}} \;-\; \underbrace{\textstyle\sum_{m\geq 2}\binom{128}{m}X_2^2X_1^m}_{\text{биномы } x_1} \pmod{\Delta^6}.$$

Используются соотношения: x_1 (16 раз, через $x_1^{128} = P_1^{16}$), x_5 (один раз), x_2 (внутри Леммы 13) и класс ≤ 4; не используются соотношение x_3, $G'' = 1$, $d_l^{N_l}$ и верхние соотношения — как и говорит абляция. Без носителя шаг (2) заканчивается на d_2^{-16}, а шаг (4) — на $-16X_2^2D_2 = -(16X_2)X_2D_2$, гарантированном лишь в Δ^5; измерено $\notin \Delta^6$ в c4b и в c4a без соотношения x_5 [компьютер].

[компьютер, `results/c5/carrier/t6.py`, c4a, $n = 6$, сертификат выполнен] каждый кусок: $128X_2^2X_1 - X_2^2(x_1^{128} - 1)$, $x_1^{128} - x_5^{1024} + 64D_3$, $X_2^2(x_5^{1024} - 1)$, $64X_2^2D_3$, $1024X_2^2X_5$, биномы x_1 и x_5, $512X_2^2$, $8D_1X_2$, $128D_3X_2$, $128D_5X_2$ и $128X_2^2X_1 - \big(X_2^2(x_5^{1024} - 1) - 64X_2^2D_3 - \sum_{m\geq 2}\binom{128}{m}X_2^2X_1^m\big)$ — все в Δ^6.

Лемма 15 [на бумаге] (где живёт w). Пусть G класса $c = k+1$, $x_i^{E_i} = P_i \in \gamma_k$, $E_j \mid A_{ij}$, $g_1 \in (\gamma_k)^{E_1}\gamma_c$, $g_2 \in (\gamma_k)^{E_2}\gamma_c$ (спаривание с множителем 1). Тогда

$$w = [x_2, x_1^{A_{12}}] = \prod_l [x_2, d_l]^{-a_l A_{12}/E_1} \qquad (P_1 = \textstyle\prod_l d_l^{-a_l}).$$

Доказательство. Для $y, z \in \gamma_k$ $[x_i, yz] = [x_i, y][x_i, z]$ и $[x_i, y^{E_i}] = [x_i^{E_i}, y] = [P_i, y] = 1$ ($[x_i, y]$ централен, γ_k абелева). Поэтому $u_{12}u_{13} = [x_1, g_1] = 1$ и $u_{23} = [x_2, h_{23}] = [x_2, g_2][x_2, h_{21}]^{-1} = [x_2, x_1^{A_{12}}]$. □

Так как $[x_2, d_l]^{E_2} = 1$, d_l-часть с $E_2 \mid a_l A_{12}/E_1$ ничего не даёт. В c4a и `c5i5` спаривание выполнено с множителем 2 (§6.11(b)), поэтому $w = [x_2, d_2]^{-32}[x_2, d_3]^{-128} = [x_2, d_2]^{-32}$: w **— это ровно d_2-часть, перенесённая носителем.** Это не совпадение (Предложение 16 ниже): часть, которую спаривание гарантирует само, имеет $E_2 \mid a_l A_{12}/E_1$ и в w исчезает; значит, для $w \neq 1$ нужна часть, которую обязательно переносить.

(b) Почему один уровень Прогоним доказательство Теоремы 14 против Δ^7 (c4a, $k = 3$). Шаги (1) и (4) проходят на уровень дальше: $64X_1^2 = 8E_1X_1^2 \in \Delta^5$ (Лемма 13 для x_1, $s = 0$: $e_2 = 1$, $E_1 \mid 8$; $e_3 = 4$) кладёт биномы x_1 в Δ^7, а $1024X_2^2X_5 = (16E_2X_2^2)X_5$ при $1024X_2^2 \in \Delta^6$ (Лемма 13, $s = 1$) лежит в Δ^7. Шаг (3) — нет: у $64X_2^2D_3 = (E_2X_2)X_2D_3$ гарантированный уровень $2 + 1 + 3 = 6$ и не больше, а 64 — это показатель, который даёт соотношение x_1 ($a_3N/E_1 = 4 \cdot 128/8$). Итак, носитель поднимает свою часть (d_2-часть) на два уровня над спариванием (с $1 + 1 + k$ до $\geq 2k + 1$), а останавливает часть, которую он не переносит. [компьютер] (Modal A10G, ref `ae122f7`, приложение `math170-n7`, `gpu/modal_batch.py spawn --ref ae122f7 --gpu a10g --input results/c5/carrier/records/c4a_pieces.jsonl.gz -- --n 7 --h 1 0 0 0 --verbose`; 91 с, замыкание 6

раундов, пик GPU 5,5 ГиБ; локальная попытка на CPU `c4a_pieces_n7.log` была остановлена после раунда 2.) $M = 19\,530$, 1 554 строки соотношений, ранг 19 524, свободных столбцов $6 = r_7$, индекс 34 830 / 34 818 / 34 758 по модулю $2^{32}/2^{30}/2^{20}$ (разности $12 = 2r_7$, $72 = 12r_7$): **сертификат выполнен**; ранг, индекс и пять w-тестов совпадают с `c4a_lin2_n7.log`. Принадлежность Δ^7 (c4a):

кусок	$\in \Delta^7$
$128X_2^2X_1$ (T_{12})	нет
$X_2^2(x_5^{1024}-1)$ (перенесённая d_2-часть)	**да**
$64X_2^2D_3$ (непереносимая d_3-часть)	**нет**
$\sum_{m\ge2}\binom{128}{m}X_2^2X_1^m$ (биномы x_1) $= -(128X_2^2X_1 - X_2^2(x_1^{128}-1))$	**да** (обе формы)
$8128X_2^2X_1^2$ (её член $m=2$)	да
$1024X_2^2X_5$; биномы x_5	да; да
$128X_2^2X_1 - \big(X_2^2(x_5^{1024}-1) - 64X_2^2D_3 - \sum_{m\ge2}\binom{128}{m}X_2^2X_1^m\big)$ (остаток тождества)	**да**
$x_1^{128} - x_5^{1024} + 64D_3$ (без множителя X_2^2; входит только как $X_2^2\cdot(\dots)\in\Delta^8$)	нет
$512X_2^2$, $1024X_2^2$, $2048X_2^2$	нет, нет, нет
$8D_1X_2$; $128D_3X_2$, $128D_5X_2$	да; нет, нет
$16X_2^2D_2$ (d_2-часть, прочитанная без носителя)	да

Итак, тождество Теоремы 14 верно по модулю Δ^7, и все его члены, кроме $64X_2^2D_3$, лежат в Δ^7: $128X_2^2X_1 \equiv -64X_2^2D_3 \pmod{\Delta^7}$, и **непереносимая d_3-часть — единственное препятствие в Δ^7 в c4a**: предсказание этого пункта подтверждено [компьютер]. $1024X_2^2 \in \Delta^6$ ($n=6$), но $2^sX_2^2 \notin \Delta^7$ при $s \le 11$: член носителя лежит в Δ^7 через $(1024X_2^2)X_5 \in \Delta^6\Delta$, как в доказательстве. Попутно: $16X_2^2D_2 \in \Delta^7$ в c4a (в c4b и без соотношения x_5 было $\notin \Delta^6$), т.е. при соотношении x_5 d_2-часть гарантирована и при прямом чтении; доказательство через Лемму 13 не записано [гипотеза: это $(16X_2)X_2D_2$ при $d_2^{16} = x_5^{-1024}$ и Лемме 13 для x_5]. Строка и лог: `results/c5/carrier/c4a_pieces_n7_a10g.out.jsonl`, `.out.log`.

(c) Условия и второй носитель **Предложение 16** [на бумаге]. Пусть $k \in \{3,4\}$, G класса $k+1$, $P_1 = \prod_l d_l^{-a_l}$, $N = A_{12}/2 = 2^\nu$. Пусть (i) для x_2 выполнена Лемма 13 с $s = k-3$; (ii) $\frac{N}{2}X_1^2 \in \Delta^{2k-2}$ (при $k=3$: $N \ge 2E_1^2$; при $k=4$: Лемма 13 для x_1 с $s=0$ и $N \ge 16E_1$); (iii) для каждого l либо

- (спаривание) $v_2(a_lN/E_1) \ge e_2 + k - 3$, либо
- (носитель) $d_l^{-a_lN/E_1} = x_\kappa^F$ для образующей x_κ и $2^kE_2 \mid F$.

Тогда $NX_2^2X_1 \in \Delta^{2k}$. *Доказательство:* Теорема 14 с этими числами: непереносимая часть есть $(\frac{a_lN}{E_1}X_2)X_2D_l$ при $2^{e_2+k-3}X_2 \in \Delta^{k-1}$; перенесённая — $X_2^2(x_\kappa^F-1)$, её первый член $(2^kE_2X_2^2)X_\kappa$ при $2^kE_2X_2^2 \in \Delta^{2k-1}$ (Лемма 13 с $s=k-3$), а у второго $v_2\binom{F}{2} \ge e_2+k-1$. □

По Лемме 15 части типа «спаривание» в w не видны (при $k \ge 3$ $v_2(a_lA_{12}/E_1) \ge e_2+k-2 \ge e_2$), так что несущая w часть P_1 обязана удовлетворять условию носителя. **Значит, Предложение 16 требует второго носителя, когда условие спаривания не выполняют две d_l-части.** Данные:

группа	k	d_2-часть x_1^N	d_3-часть	$NX_2^2X_1$ (предсказание)	измерено
c4b	3	d_2^{-16}, без носителя, $v_2 = 4 < 6$	d_3^{-64}, $6 \ge 6$	не гарантировано в Δ^6	$\notin \Delta^6$
c4a	3	носитель x_5^{1024}, $2^3E_2 = 512 \mid 1024$	$6 \ge 6$	$\in \Delta^6$	$\in \Delta^6$
c4a, цель Δ^7	3	носитель (Лемма 13, $s=1$)	$6 < 7$	не гарантировано в Δ^7	$\notin \Delta^7$ (§6.11); вне Δ^7 только $64X_2^2D$
`c5i5`	4	носитель, $2^4E_2 = 1024 \mid 1024$	$6 < 7$	не гарантировано в Δ^8	$\notin \Delta^8$ (§6.11(g))
`c5i6` (ниже)	4	носитель x_5	носитель x_6^{1024}	$\in \Delta^8$	— ($d=6$, $n=8$ недоступно)

Так что гипотезу H из §6.11 следует читать так: носитель целиком чинит одну d_l-часть; сколько уровней это даёт для w, решает худшая *непереносимая* часть, а она стоит на $\Delta^{k+3} = \Delta^{c+2}$ в (3.12) и `c5i5`. Это и есть наблюдённое «один носитель — один уровень».

(d) Кандидаты класса 5, построенные в GAP [компьютер, `scripts/c5_carrier_gap.sh` = `gap/c5_ep.g` на записях `I5cand`, Docker, локально; журналы `results/c5/carrier/ep_*.log`]

имя	d	изменение относительно `c5i5`	множитель (по построению)	Предл. 16 для $128X_2^2X_1$
`c5i5` (§6.7)	5	—	2	d_3-часть не проходит
`c5i6`	6	$+x_6$, соотношение $x_6^{1024}d_3^{64}$ (второй носитель)	2	выполнено
`c5m4`	5	$P_1 = d_2^{-4}d_3^{-8}$, $P_3 = d_1^{-4}d_2^{-32}$, $x_5^{1024} = d_2^{-64}$, $A_{23} = 4E_3$	4	выполнено
`c5m4b`	5	$P_1 = d_2^{-1}d_3^{-8}$, $P_2 = d_1^4d_3^{64}d_5^{-16}$, $P_3 = d_1^8d_2^{-16}$, $A_{13} = A_{23} = 2E_3$	4	выполнено
`v1`	5	только $A_{13} = 2E_3$	—	не выполнено
`v2`	5	только $P_1 = d_2^{-1}d_3^{-8}$	—	выполнено
`v3`	5	только $P_2 = d_1^4d_3^{64}d_5^{-16}$	—	не выполнено
`v4`	5	только $P_3 = d_1^8d_2^{-16}$	—	не выполнено

(Остальные соотношения как в `c5i5`: d_1^{16}, d_2^{128}, d_5^{2048}, два верхних. Множители — по построению, из точных g_i в γ_4: для `c5m4b` $g_1 = d_1^{32}d_2^{-32}d_3^{256}d_5^{-64} \in (\gamma_4)^{32}$, $g_2 = d_1^{16}d_3^{256} \in (\gamma_4)^{256}$, $g_3 = 1$; `c5_ep.g` останавливается на «$w = 1$ в M» раньше своего отчёта о множителях. Каждый прогон — 1–2 ч GAP/nq в Docker на нагруженной машине. `c5m4x6` = `c5m4` $+x_6^{1024}d_3^{128}$ и `c5m4b6` = `c5m4b` $+x_6^{1024}d_3^{128}$ остановлены вручную: копредставление, содержащее все соотношения другого на большем числе образующих, переводит $w \mapsto w$, так что и там $w = 1$ [на бумаге].)

Что говорит таблица:

- `c5m4` [на бумаге, Лемма 15]: при множителе ≥ 1 $w = [x_2, P_1^{32}] = [x_2, d_2]^{-128}[x_2, d_3]^{-256}$ и $[x_2, d_l]^{64} = 1$: $w = 1$ вынуждено. Увеличение a_2 ради спаривания убивает несущую w часть — Лемма 15 в действии.
- `c5m4b` сохраняет $a_2 = 1$, и Лемма 15 даёт $w = [x_2, d_2]^{-32}$, как в `c5i5`; всё равно $w = 1$: здесь падает порядок $[x_2, d_2]$. Варианты с одним изменением показывают, что **каждое** из четырёх изменений делает это само по себе; и так же второй носитель `c5i6`, не трогающий ни P_1, ни P_2, ни P_3, ни A. [гипотеза] Порядок $[x_2, d_2]$ в γ_5 связан с d_3-частью через соотношение x_1 (в кольце Ли $[x_1^8 d_2 d_3^{a_3}, x_2]$ связывает $[d_2, x_2]$ с $a_3[d_3, x_2]$ и с $8[x_1, x_2]$), поэтому, делая d_3-часть безвредной для T_{12} — носителем или $a_3 = 8$, — мы делаем тривиальным и $[x_2, d_2]^{32}$. Не доказано.
- Плотные проверки: кандидата с $w \neq 1$ нет, поэтому не запускались ($\dim A_6 = 9\,330$ при $d = 6$ был бы посилен CPU, $\dim A_7 = 55\,986$ и $\dim A_8 = 335\,923$ — нет).

(e) Статус

- Доказано [на бумаге]: Лемма 13, Теорема 14 (тождество для c4a), Лемма 15, Предложение 16; точный вид Леммы 13 при $n = 5$ [компьютер, над $\mathbb{Z}$].
- Вычислено [компьютер, сертифицировано]: каждый кусок Теоремы 14 в Δ^6 (c4a), $e_1 \notin \Delta^6$ в c4b и без соотношения носителя, абляция, уровни 2^sX_2 и $2^sX_2^2$; GAP: $w = 1$ во всех семи новых копредставлениях. При $n = 7$ (A10G): все куски, кроме $64X_2^2D_3$, в Δ^7, т.е. d_3-часть — единственное препятствие в Δ^7 в c4a ((b)).
- Проверено независимым проверяльщиком (D5-21; `results/cert/carrier/`, `cert/README.md`): 22 утверждения, 288 принадлежностей и 213 непринадлежностей, все VERIFIED программой `cert/check_in.c`; 17 утверждений с принадлежностями есть и в трассах, а `carrier_c4aT4` при $n = 4$ (все 26 членов) проигрывает ядро Lean (§3.2(f)). Это все куски (a) и (b) при $n = 4, \ldots, 7$ (c4a), уровни 2^sX_2, $2^sX_2^2$ и тесты `t1`, `t3`–`t5`, прогон c4b, абляция (без соотношений $d_l^{N_l}$ и верхних, метабелевых или соотношений $\ker\varphi$: по-прежнему $128X_2^2X_1 \in \Delta^6$; без соотношений x_1, x_2, x_3: непринадлежности), а также непринадлежности и индексы копредставлений без соотношения x_5. Ранг берётся из отображения сумм показателей φ ($x_4 \mapsto 1$), а не из h. Принадлежность заявляется, только если $G/G' \cong \mathbb{Z} \oplus T$, где T — 2-группа, и тест обнуляется в $\mathbb{Z}[t^{\pm1}]/(t-1)^n$ при $g \mapsto t^{\varphi(g)}$. Тогда при $\bar{G} = G/\gamma_n(G)$ (то же $\mathbb{Z}[G]/\Delta^n$) тест лежит в $\Delta(K)\mathbb{Z}[\bar{G}] + \Delta^n$, $K = \ker\varphi$ на $\bar{G}$, и убивается степенью двойки: каждый $\gamma_k/\gamma_{k+1}(\bar{G})$, $k \geq 2$, — образ $(G/G')^{\otimes k}$, где часть $\mathbb{Z} \otimes \cdots \otimes \mathbb{Z}$ переходит в $[x_4, \ldots, x_4] = 1$, так что он убивается $\exp T$ и K — конечная 2-группа [на бумаге]. Данные nq не нужны. Точный вид Леммы 13 при $n = 5$ перепроверен над $\mathbb{Z}$ программой `cert/check_identity.py`. Независимо не проверены: принадлежности там, где свободный ранг G/G' равен 2 (без соотношения x_1, x_2, x_3 или x_5; ограничителю нужен ранг 1), а также $w = 1$ от GAP в семи копредставлениях из (d).
- Гипотезы: что в семействе (3.12) с $t = [x_4, x_3, x_3]$ условие Предложения 16 для T_{12} в классе 5 и $w \neq 1$ исключают друг друга ((d)).

- Не сделано: копредставление класса 5 с множителем 4, обеими починенными частями T_{12} и $w \neq 1$. Следующие кандидаты должны сохранить $P_1 = d_2^{-1}d_3^{-4}$ и A из `c5i5` и чинить d_3-часть образующей, не входящей в лиево соотношение x_1 с x_2 (например, носитель для $d_3^{\S 4}$ в соотношении x_3 вместо нового степенного соотношения) — не проверено.

Перезапуск.

```
# проверки (около 15 мин на нагруженной машине)
python3 scripts/c5_carrier_tests.py results/c5/presentation/records/c4a_lin2.jsonl.gz OUT.jsonl.gz --replace
python3 gpu/howell_gpu.py --device numpy plateau --input OUT.jsonl.gz --n 6 --h 1 0 0 0 --steps host
python3 results/c5/carrier/summ.py LOG    # печатает таблицу
# абляция
scripts/c5_carrier_ablate.sh BASE TESTS.py 6 TAG "4-8" "54-303" "0" ...
# минимальные строки и точное тождество
python3 scripts/c5_carrier_rows.py results/c5/presentation/records/c4a_lin2.jsonl.gz --n 5 --target '512*X2*X
python3 scripts/c5_carrier_exact.py OUT.json EX.json
sage scripts/c5_carrier_solve.sage EX.json
# GAP
scripts/c5_carrier_gap.sh c5i6 c5m4 c5m4b v1 v2 v3 v4
```

Расход Modal на §6.12: $0,05 (один вызов A10G, 91 с счёта, приложение `math170-n7`, остановлено; `modal billing report`: $0,054); всё остальное локально (замыкания на CPU, GAP в Docker, Sage).

6.13. D5-11: J_n без плотной формы — стандартные мономы, уровень за уровнем

Вопрос: проверить $w - 1 \in J_n$ для $d = 6$ образующих при $n = 7, 8$, где плотная форма §6.5 имеет $M = 55\,986$ и $335\,922$ столбца (≈ 450 ГБ в `uint32` при $n = 8$). Здесь J_n строится по степеням, в столбцах только *мономов-кандидатов*; на всех группах, что у нас есть, получаются те же индекс, образующие и принадлежности, что у плотного замыкания, а полная приведённая форма Хауэлла, восстановленная по ней, совпадает с плотной байт в байт. Код: `gpu/stdmono.py` (и `gpu/sparse_howell.c` для `--sparse`, `gpu/modal_stdmono.py` для Modal). Данные: `results/nondense/`.

Итог.

- [на бумаге] $J_k = \mathrm{span}\{\mu(r) - 1\} + J_{k-1}a + aJ_{k-1}$ (лемма N1): одна форма Хауэлла на уровень, без раундов замыкания. Слово, у которого префикс или суффикс не стандартный моном предыдущего уровня, переписывается фиксированным правилом в более поздние столбцы (проекция π_k с ядром в J_k), так что столбцами остаются только кандидаты C_k; образующие $\pi_k(J_k)$ — соотношения, оба сдвига кручёных строк и немного «перекрытий» (лемма N2).
- [компьютер] Проверка: c4a $n = 5, 6, 7$, `c5i5` $n = 6, 7, 8$, M из §6.10 (q5) $n = 6, 7, 8$: индекс по модулю $2^{32}/2^{30}/2^{20}$, образующие, все tests и lintests — как в плотных прогонах; SHA-256 полной приведённой формы совпадает с плотной для c4a $n = 5, 6, 7$ и `c5i5` $n = 6, 7$. `c5i5` при $n = 8$ воспроизводит §6.11(g) на A10G за 5,4 мин (плотно: 26 мин на A100), с $45\,530$ столбцами вместо $97\,655$.
- [компьютер] Приведённая форма в этих столбцах очень разрежена: 0,57 млн ненулей на $45\,530$ строк (`c5i5`, $n = 8$), и строки-образующие тоже (14 ненулей на сдвиг кручёной строки). Это использует `--sparse` (по строке, на C).
- [компьютер] `--sparse` при $n = 8$ (`c5i5`, A10G): снова ответы §6.11(g), за 126 с вычислений (155 с с контейнером), пик GPU 2,0 ГиБ, пик хоста 8,7 ГиБ; в форме $573\,467$ ненулей (у канонической $565\,950$) — заполнения нет.
- [оценка] $d = 6$: $n = 7$ — легко (около 34–41 тыс. столбцов; минуты на одном GPU, плотно или разреженно). $n = 8$: 154–203 тыс. столбцов; плотная форма (48–82 ГБ) на один GPU 80 ГБ не помещается, а `--sparse` помещается: около 25–35 мин на одном A10G (15–20 мин на A100), память GPU $\approx$ 8–12 ГБ, хоста $\approx$ 20–25 ГБ. Подробности в (d).

(a) Структура и варианты, с размерами Столбцы по возрастанию степени, как везде в B. Используемые факты [на бумаге, §3.4]: форма Хауэлла J_n, обрезанная до степеней $< k$, есть форма Хауэлла J_k; столбцы без единичного опорного (*стандартные мономы* S_k) замкнуты относительно префиксов и суффиксов (если у u единичный опорный, то $u - \mathrm{NF}(u) \in J$ с $\mathrm{NF}(u)$ на более поздних столбцах, и умножение на a_j с любой стороны

сохраняет «позже»); $\mathrm{NF}_k(u)$ лежит в стандартных мономах степени $\geq \deg u$. Число стандартных мономов не меньше $\dim_{\mathbb{F}_2} \Delta_{\mathbb{F}_2}/\Delta^n_{\mathbb{F}_2}$ (`generators`, Дженнингс, §6.4) — нижняя граница для любого мономиального базиса.

вариант	столбцов, `c5i5` $n = 8$ (измерено)	$d = 6$, $n = 8$ (модель из (d))
плотно (§6.5)	97 655	335 922
по уровням во всех столбцах (только лемма N1)	97 655	335 922
сжата только старшая степень	19 530 + 33 709	$\approx$ 56 000 + 130 000
кандидаты C_n (выбрано)	45 530	154 000–203 000
нижняя граница: образующие	44 370	$\geq$ около 150 000
кандидаты, разреженная форма	45 530	то же
подпространство, достижимое из $w-1$, или двойственное	—	выигрыша нет: для принадле
метабелевы соотношения Γ''	—	уже учтены: это соотношени

Порядок степеней «сверху вниз» тоже не помогает: число образующих — свойство фактора, а не порядка. Поэтому выбран метод столбцов-кандидатов по уровням, плотный или разреженный.

(b) Метод Обозначения: в A_k степени $< k$; $J_{k-1} \subseteq A_{k-1}$ — обрезка J_k (§3.4); S_k, NF_k как выше.

Лемма N1 [на бумаге]. $J_k = \mathrm{span}\{\mu(r) - 1 : r \in R_0\} + \sum_j (J_{k-1}a_j + a_j J_{k-1})$, произведения в A_k (и по модулю 2^E). *Доказательство.* Для $z \in J_{k-1}$ возьмём подъём $\tilde{z} \in J_k$; $\tilde{z}a_j = za_j$ в A_k, так как часть $\tilde{z}$ степени $k-1$ в произведении умирает; отсюда «$\supseteq$». Обратно, J_k натянуто на $u(\mu(r)-1)v$ с мономами u, v; если $v = v'a_j$, то $u(\mu(r)-1)v = \big(u(\mu(r)-1)v'\big)a_j$, скобка лежит в J_k, её обрезка — в J_{k-1}; слева так же. □

Кандидаты и проекция. C_k: слова степени 1 и слова v степени $p \geq 2$ с $v[:-1], v[1:] \in S_{k-1}$. Так как $S_k \cap (\deg < k-1) = S_{k-1}$ и S_k замкнуто относительно префиксов и суффиксов, $S_k \subseteq C_k$. Для $v \notin C_k$: если $u = v[:-1] \notin S_{k-1}$, то $v \mapsto \mathrm{NF}_{k-1}(u)\, a_l$ (правило префикса); иначе $v = a_f u'$ с $u' \notin S_{k-1}$, $v \mapsto a_f\, \mathrm{NF}_{k-1}(u')$ (правило суффикса). Каждый шаг — сравнение по модулю J_k ($u - \mathrm{NF}_{k-1}(u)$ — обрезка элемента J_k; разница в степени $k-1$ умирает после сдвига) и уводит в строго более поздние столбцы; π_k — результат. Значит, $x - \pi_k(x) \in J_k$, π_k тождественна на C_k, и

$$x \in J_k \iff \pi_k(x) \in \pi_k(J_k), \qquad A_k^+/J_k \cong \mathbb{Z}^{C_k}/\pi_k(J_k)$$

— индекс, свободные столбцы и `generators` те же, что у плотной формы, и сертификат §6.4 применяется без изменений.

Лемма N2 [на бумаге]. $\pi_k(J_k)$ порождено π_k от: (i) $\mu(r)-1$; (ii) ta_j, $a_j t$ для строк t формы Хауэлла $\pi_{k-1}(J_{k-1})$ с неединичным опорным; (iii) *перекрытий* $a_j(u - \mathrm{NF}_{k-1}(u))$ для $u \notin S_{k-1}$ с $u[:-1] \in S_{k-1}$ и $a_j u[:-1] \notin S_{k-1}$. *Доказательство.* По N1, с порождающим множеством $\{g_u = u - \mathrm{NF}_{k-1}(u) : u \notin S_{k-1}\} \cup \{\text{кручёные строки}\}$ для J_{k-1} (приведённая форма Хауэлла во всех столбцах). $\pi_k(g_u a_j) = 0$ по правилу префикса. $\pi_k(a_j g_u) = 0$, если $a_j u[:-1] \in S_{k-1}$ (к $a_j u$ применяется правило суффикса). Если $u = u'a_l$ с $u' \notin S_{k-1}$: $z = g_u - g_{u'}a_l \in J_{k-1}$ лежит на столбцах после u, и $a_j g_u = \big(a_j g_{u'}\big)a_l + a_j z$ в A_k; первое слагаемое — правый сдвиг элемента J_{k-1} (его π_k в линейной оболочке (ii)), а z — комбинация g_x, $x > u$, и кручёных строк (свойство Хауэлла); индукция вниз по u. □

Дальше H_k — форма Хауэлла этих строк в столбцах C_k, а $\mathrm{NF}_k(u)$ для $u \notin S_k$ — это $\pi_k(u)$, приведённое по H_k; оно хранится матрицами $T_k[p]$ (строки: нестандартные слова степени p, столбцы: кандидаты степени $\geq p$) для правил префикса и суффикса уровня $k+1$. Свойство обрезки проверяется на каждом уровне (опорные и их нормы ниже старшей степени совпадают с H_{k-1}).

Реализация [компьютер]. `gpu/stdmono.py run` (те же поля вывода, что у `howell_gpu.py batch`, плюс `columns` и статистика по уровням): соотношения и тесты строятся один раз в A_n (для степеней длинных слов — биномиальное разложение: $n-2$ произведения вместо двоичного возведения) и обрезаются на каждом уровне; π_k — две GEMM на букву с T_{k-1}, повторяемые, пока не останется столбцов вне кандидатов; форма Хауэлла и её образы по модулю $2^{30}, 2^{20}, 2$ — через `howell_gpu.py` (numpy или cupy), или с `--sparse` — через `gpu/sparse_howell.c`: строки вставляются по одной, сначала с самым поздним ведущим столбцом, каждая новая опорная строка приводится по уже имеющимся опорным (в порядке порождения и без приведения заполнение было в 12 раз больше: c4a $n = 6$, 210 497 ненулей против 17 162; $n = 7$ — 16,9 млн и 23 мин во вставке на C, остановлено). `--digest` восстанавливает приведённую форму во всех M столбцах (строки H_n и $u + \mathrm{reduce}(-\pi_n(u))$ для остальных слов) и печатает `howell_gpu.digest`. 400 случайных матриц: форма на C и `howell_gpu.howell` совпадают по опорным, индексу, образам и принадлежности.

(c) Проверка против плотного замыкания [компьютер, сертифицировано] (D5-21, 01.11.2026; принадлежности есть и в трассах, §3.2(f)). Ответы c4a при $n = 5, 6, 7$ и `c5i5` при $n = 6, 7, 8$ ниже (все тесты и линейные тесты, индекс по модулю 2^{32}) подтверждены программой `cert/check_in.c`, не делящей кода с этим методом: своё разреженное замыкание по уровням во всех M столбцах, проверенные замкнутость и свойство Хауэлла, точность сверена с r_n (`results/cert/{c4a,c5i5}_n*_E32.claim`, записаны по этим прогонам скриптом `cert/make_claims.py`). `c5i5` при $n = 8$: 97 648 строк, 4,9 млн ненулевых, 32 с и 190 МБ на одном ядре kimmy. На доверии: копредставления, а для принадлежности — что порядок элемента (для линейного теста — его образа в Δ/Δ^n) есть степень двойки. Последнее верно, потому что γ_2 — конечная 2-группа ($h_k = 0$ при $k \geq 2$; nq): тесты лежат в γ_c, а линейные тесты составлены только из x_1, x_2, x_3, и $x_i^{E_i} \in \gamma_k$ (§6.12), так что порядок каждого g_k — степень двойки (`cert/README.md`).

[компьютер] Во всех прогонах сертификат выполнен (свободных столбцов = r_n, разности индексов $2r_n$, $12r_n$). «Плотно» — прежние прогоны §6.10, §6.11 или сделанные здесь (`c5i5` $n = 6$ на локальном CPU, $n = 7$ на A10G, `results/nondense/a10g_n7/c5i5_dense_n7.*`).

группа	n	M	столбцов	индекс $2^{32}/2^{30}/2^{20}$	образующих	tests, lintests	SHA-2
c4a	5	780	780	3 311 / 3 303 / 3 263	619	=	`f1a381`
c4a	6	3 905	2 626	10 834 / 10 824 / 10 774	2 325	=	`56f3da`
c4a	7	19 530	8 851	34 830 / 34 818 / 34 758	8 276	=	`0d3edc`
`c5i5`	6	3 905	3 456	13 717 / 13 707 / 13 657	3 000	=	`8b903a`
`c5i5`	7	19 530	12 464	48 533 / 48 521 / 48 461	11 755	=	`dca6a0`
`c5i5`	8	97 655	45 530	169 278 / 169 264 / 169 194	44 370	= (§6.11(g))	—
q5 (M из §6.10)	6, 7	3 905; 19 530	3 456; 14 165	14 722 / …; 54 097 / 54 085 / 54 025	3 214; 13 039	=	—
q5	8	97 655	52 973	195 779 / 195 765 / 195 695	51 004	=	—

(«=»: так же, как в плотном прогоне; время — замыкание, соответственно весь прогон `stdmono.py` со всеми уровнями и сводкой.) `--sparse` даёт те же индекс, образующие и принадлежности для c4a $n = 5, 6, 7$ и `c5i5` $n = 7$ (локальный CPU, `results/nondense/cpu_sparse/`). Его форма: c4a — 17 162 ненулей при $n = 6$, 101 033 при $n = 7$ (11 на строку); `c5i5` $n = 7$ — 96 823 (8 на строку; у канонической формы 95 107). Вставка на C заняла 1,2 с, 17 с и 7,8 с на сильно загруженной машине (проекция numpy-GEMM там — 65–76 мин; на A10G это секунды).

`--sparse` при $n = 8$ [компьютер] (`c5i5`, Modal A10G, 4 CPU и 16 ГиБ хоста; `results/nondense/a10g_sparse_n8/`): индекс 169 278 / 169 264 / 169 194, образующих 44 370, свободных столбцов $7 = r_8$, список in_D из §6.11(g) поэлементно; сертификат выполнен. Всего 126 с (соотношения и тесты в A_8 — 22 с; уровень 8: проекция 44,5 с, вставка на C 23,8 с; сводка с образами по модулю $2^{30}, 2^{20}, 2$ — 14 с), пик GPU 2,0 ГиБ, пик хоста 8,7 ГиБ (из них 3,1 ГБ — соотношения в A_8). В 129 000 строках-образующих уровня 8 — 17,0 млн ненулей (соотношения 9,5 млн, сдвиги кручёных строк 5,9 млн, перекрытия 1,6 млн); в итоговой форме 573 467 ненулей, на 1,3 % больше канонических 565 950. Значит, при $n = 8$ вставка снизу вверх с приведением хвоста практически не даёт заполнения, как и при $n \leq 7$.

Побочный результат [компьютер]: `c5i5` при $n = 7$ (в §6.11(g) не измерялось): $w, S, XV, B_{12}, \sum B$, chk $\in \Delta^7$, $B_{13}, B_{23} \notin \Delta^7$, $128X_2X_1^2, 128X_2^2X_1 \in \Delta^7$. Вместе с (g): $128X_2^2X_1 \in \Delta^7 \setminus \Delta^8$ — на один уровень выше гарантированного $\Delta^{c+1} = \Delta^6$ (лемма 10), как и говорит гипотеза H (как x_5 в классе 4).

(d) Ресурсы для $d = 6$ *Модель размеров.* Стандартные мономы по степеням, измерено: `c5i5` 5, 25, 125, 580, 2279, 8807, 32842; q5 5, 25, 125, 580, 2497, 9898, 38302 (степени 1..7), коэффициенты роста 4,6 → 3,7. Для группы того же вида на шести образующих (§6.11(e)3: ещё один носитель) коэффициенты взяты $\times 6/5$ или $+1$ (степени ≤ 3 свободны), а старшие кандидаты — 1,03× старших стандартных мономов, как измерено. Тогда $C_7 \approx 34\,000$–$41\,000$ (0,61–0,74 M), $C_8 \approx 154\,000$–$203\,000$ (0,46–0,60 M). Это модель, а не измерение: у настоящей группы кручения может быть больше или меньше.

Стоимость, пересчитанная с измеренных прогонов (`c5i5`/q5 при $n = 8$: проекция 26,5 с на A100 и 44,5 с на A10G, плотный Хауэлл 50 с на A100, разреженная вставка 23,8 с, для 45 530 столбцов и 129 000 строк-образующих; проекция — $\approx 7 \cdot 10^{12}$ точных умножений-сложений в секунду на A100; для $d = 6$ она растёт в ≈ 31 раз: строк в 3,5 раза больше, работы на строку в 9 раз):

	$d = 6, n = 7$	$d = 6, n = 8$
столбцов (плотное M)	34–41 тыс. (55 986)	154–203 тыс. (335 922)

	$d = 6, n = 7$	$d = 6, n = 8$
строк-образующих старшего уровня	$\approx 90\,000$	$\approx 420\,000$–$500\,000$
плотная форма этого метода	2,3–3,4 ГБ; пик $\lesssim 25$ ГБ	48–82 ГБ; пик в разы больше: **на о**
плотный метод §6.5	6,3 ГБ; помещается на A100	226 ГБ: нет
проекция π_n (плотная GEMM с T_{n-1})	$\approx 3 \cdot 10^{13}$: секунды	$(5–8) \cdot 10^{15}$: 12–19 мин A100, ≈ 23 ми
Хауэлл, плотно	$\approx 3 \cdot 10^{13}$: 0,5 мин A100, 1–2 мин A10G; CPU ≈ 7 ч	—
Хауэлл, `--sparse`	секунды (C, одно ядро)	форма 2–4 млн ненулей (13–20 на с
память GPU, `--sparse`	$\lesssim 3$ ГБ	≈ 8–12 ГБ (2,0 ГиБ измерено при d
память хоста	соотношения в A_7: 2 ГБ	≈ 20–25 ГБ (8,7 ГиБ измерено при d
итого	**минуты на A10G** (и CPU, часы)	**`--sparse`: около 25–35 мин на одн**

H100: та же память (80 ГБ), в $\approx 1{,}5$–2 раза быстрее. CPU для $n = 8$: проекция плотной GEMM — $\sim 10^{16}$ точных операций (недели); с разреженными таблицами T (0,6 % ненулей при $n = 6$) это были бы часы — не реализовано.

(e) Что осталось

1. Сделано: у `--sparse` при $n = 8$ заполнения нет (см. выше). Для $d = 6$, $n = 8$ теперь не хватает группы, а не метода.
2. Проекция с разреженными T_{n-1} (CSR в cupyx/scipy): основная стоимость при $d = 6$, $n = 8$, и она открыла бы путь через CPU.
3. Группа на шести образующих для проверки (§6.11(e)3): не построена; модель из (d) хороша ровно настолько, насколько аналогия.
4. Хранение по модулю 2^{16} (`uint16`): вдвое меньше плотная форма; сертификат тогда берётся при $E = 16$ с $E_2 = 15, 14$ и выполняется только при показателе кручения $\leq 2^{14}$ (наибольший порядок в Δ/Δ^2 у групп вида (3.12) — 2^{14}; выше не проверялось).

Перезапуск.

```
# --device numpy на CPU
python3 gpu/stdmono.py run --input results/c5/presentation/records/c5i5_lin.jsonl.gz --n 6 7 8 --h 1 0 0 0 0
# на Modal
modal deploy gpu/modal_stdmono.py
python gpu/modal_stdmono.py a10g|a100 '<команда с $OUT>'
modal volume get math171-nondense <dir> .
```

Расход Modal на §6.13: \$1,69 по `modal billing report` (приложение `math171-nondense`: проверка на A10G при $n \leq 7$ вместе с плотным `c5i5` $n = 7$; `c5i5` и q5 при $n = 8$ на A10G (q5 там не хватило памяти) и на A100; прогон статистики; первая попытка `--sparse` без приведения хвоста, остановлена примерно через 25 мин); затем \$0,09 на `--sparse` при $n = 8$ (один прогон A10G, 155 с; то же приложение, заново развёрнутое). Всего \$1,78. Всё остановлено.

7. Литература и известен ли метод

Знак † — ссылка не сверена с источником. DOI проверены через API Crossref (название, журнал, том, страницы).

- Формула $D_n(F/R) = F \cap (1 + \mathfrak{r} + \mathfrak{f}^n)$ и вложение Магнуса — классика теории размерных подгрупп: [MP09] R. Mikhailov, I. B. S. Passi, *Lower Central and Dimension Series of Groups*, Lecture Notes in Math. 1952 (2009), doi:10.1007/978-3-540-85818-8, гл. 1 (точный параграф не сверен) †; N. Gupta, *Free group rings*, Contemp. Math. 66 (1987), doi:10.1090/conm/066 †. Усечённый изоморфизм (a) — частный случай теоремы Магнуса (W. Magnus, 1937 †).
- [BM23] L. Bartholdi, R. Mikhailov, *Group and Lie algebra filtrations and homotopy groups of spheres*, arXiv:1805.10894 (проверено через API arXiv); журнальная версия J. Topology 16 (2023) 822–853, doi:10.1112/topo.12301 †. Приложение `3group.gap` — pc-копредставление 3-группы порядка 3^{494} на 7 порождающих с проверкой соотношений и класса в GAP. Вычислений в групповом кольце там нет: доказательство теоретическое.
- [IMP26] V. Ionin, R. Mikhailov, T. Petrov, *Dimension subrings of Lie rings*, arXiv:2610.02042 (проверено через API arXiv).
- [HMP07] M. Hartl, R. Mikhailov, I. B. S. Passi, *Dimension quotients*, arXiv:0803.3290 (проверено через API arXiv, он даёт J. Indian Math. Soc. (N.S.), специальный том к столетию IMS (1907–2007), 63–107). Явного копредставления группы Рипса там нет.

- ModIsom (B. Eick, пакет GAP): `NilpotentQuotientOfFpAlgebra` — нильпотентный фактор конечно представленной ассоциативной алгебры. В документации он работает над конечным полем (пример и поле `fld` в ответе), то есть даёт $\mathbb{F}_p[G]/\Delta_{\mathbb{F}_p}^n$ — ряд Дженнингса M_n, а не целочисленные D_n. Поддержка $\mathbb{Z}$ по исходникам не проверена †. Алгоритм: B. Eick, *Computing nilpotent quotients of associative algebras and algebras satisfying a polynomial identity*, IJAC 21 (2011), doi:10.1142/S0218196711006649 †.
- LAGUNA — модулярные групповые алгебры $\mathbb{F}_p[G]$, тоже Дженнингс, не $\mathbb{Z}$.
- Тот же метод реализован независимо (не опубликовано); сверка — §5.

Вывод. Тождество $\mathbb{Z}[G]/\Delta^n \cong A_n/J_n$ классическое. Его вычислительного применения к целочисленным D_n — линейной алгебры над $\mathbb{Z}/p^E$ или $\mathbb{Z}$ в A_n размера, не зависящего от $|G|$, — мы в найденной литературе не встретили. Поиск не был исчерпывающим.